University of Essex

Master of Science Thesis

Measuring Technical Debt in Mission Critical Trading Systems

A Service Level Quantitative Framework

Author:
Tobias Zeier

Student ID:
12696372

Supervisor:
Dr Zahid Ullah

Co-Supervisor:
Doug Millward

A thesis submitted in fulfilment of the requirements for the degree of Master of Science

in the

Enterprise IT Management Programme
Computing Department

University of Essex

2 August 2026

Declaration of Authorship

I confirm that this assignment:

- A: Is my own work and there is no involvement from a third party. Where someone else's ideas are used, I have referenced and paraphrased those materials. I have read and understood the University of Essex Online Integrity guidelines and declare that this assignment conforms to the rules and regulations.
- B: Does not use documentation or data, which is not freely accessible to the public without permission. In addition, where information may identify stakeholders (participants, employees, patients, etc) this has been anonymised.
- I have made use of AI tools in the preparation of this thesis. The tools employed and the nature of their use are detailed below:
  - **Tools used:** Claude AI, Google Gemini, and Grammarly.
  - **Purpose:** I used these tools to improve the clarity, grammar, consistency of the writing, and debugging of software code.
  - **Scope:** AI assistance was used for proofreading across the full thesis and for early-stage planning of the chapter structure.

Abstract

Measuring Technical Debt in Mission Critical Trading Systems

Tobias Zeier – 12696372

Word count: 13,675

Technical debt in mission-critical trading IT accumulates largely outside source code. It builds up in infrastructure lifecycle exposure, patch delay and operational instability, where the dominant code-centric measurement tools cannot observe it. This thesis develops and evaluates a quantitative framework for prioritising technical debt at the service level using operational data alone. Following a Design Science approach, five indicators (incident frequency, mean time to recover, change failure rate, patch recency and unsupported component months) were extracted from two years of live ITSM and CMDB records at a regulated Swiss financial trading institution. They were combined in a TOPSIS model that ranks six production services by debt priority under two weight configurations and across two observation years. The evaluation tests the ranking on five fronts: its stability under weight and temporal variation, its convergence with SAW and VIKOR, its divergence from an incident-only baseline, and its structural rigour under service removal. The last is addressed through a rank-reversal-free absolute-mode TOPSIS variant. Scored with that variant across 20,000 sampled weightings, the lowest-priority service is near-invariant to the weighting, while the single highest priority is weight-dependent within a stable three-service cluster. The multi-criteria design also surfaces infrastructure lifecycle debt that an incident count misses. Method disagreement, in turn, localises to that high-priority cluster, exposing a substantive choice about whether debt dimensions are compensatory. To the best of current knowledge, this is the first application of TOPSIS to live service management records for technical debt prioritisation in financial trading IT, and it contributes an auditable evaluation logic suited to regulated environments. The rankings are ordinal decision-support signals rather than measurements of debt. External and longitudinal validation are identified as the principal directions for future work.

Keywords: technical debt, technical debt prioritisation, TOPSIS, multi-criteria decision-making, MCDM, trading IT, ITSM, CMDB, DORA metrics, composite score

Acknowledgements

I would like to express my sincere gratitude to Dr Zahid Ullah for his expert guidance, patience, and continued support throughout this research project. His insightful feedback and encouragement were invaluable in shaping this work.

I am equally grateful to Doug Millward for his thoughtful co-supervision during the early stages of this thesis and his practical perspectives on the subject matter.

I would also like to thank the University of Essex for providing an excellent academic environment and the resources necessary to complete this degree.

Finally, I extend my heartfelt thanks to my colleagues, friends, and family for their understanding and encouragement throughout this journey.

Table of contents

List of Figures

List of Tables

List of Abbreviations

AHP	Analytic Hierarchy Process
BCS	British Computer Society
CFR	Change Failure Rate
CI/CD	Continuous Integration / Continuous Delivery
CMDB	Configuration Management Database
COTS	Commercial Off the Shelf
DORA	DevOps Research and Assessment
DSR	Design Science Research
FINMA	Swiss Financial Market Supervisory Authority
FADP	Federal Act on Data Protection
IT	Information Technology
ITIL	IT Infrastructure Library
ITSM	IT Service Management
MCDA	Multi-Criteria Decision-Analysis
MCDM	Multi-Criteria Decision-Making
MTTR	Mean Time to Recover
RQ	Research Question
SAW	Simple Additive Weighting
SLA	Service Level Agreement
SQALE	Software Quality Assessment based on Lifecycle Expectations
TD	Technical Debt
TOPSIS	Technique for Order of Preference by Similarity to Ideal Solution
VIKOR	VlseKriterijumska Optimizacija I Kompromisno Resenje

List of Symbols

C_i	TOPSIS closeness coefficient
S_i^+	Euclidean distance from positive ideal solution
S_i^-	Euclidean distance from negative ideal solution
x_{ij}	Raw indicator value for service i, criterion j
r_{ij}	Vector-normalised indicator value
v_{ij}	Weighted normalised indicator value
w_j	Criterion weight
u_j	Fixed upper bound (ceiling) for criterion j in absolute mode
r_{ij}^{abs}	Absolute-mode normalised value
m	Number of services evaluated
n	Number of indicators
τ	Kendall's tau, a ranking correlation coefficient

1 Introduction

Technical debt (TD) is not an abstract software quality concern. In mission-critical trading IT, it translates into delayed recovery, accumulating infrastructure risk and governance blind spots. The sections below establish why the absence of a quantitative, service-level measurement mechanism represents a specific and costly gap, and how this thesis addresses it.

1.1 Background and Motivation

Technical debt describes the implied future cost of accepting short-term technical compromises over more robust solutions (Cunningham, 1992). The concept has since expanded from source code quality to the architectural, design, test and documentation dimensions of complex IT systems (Kruchten, Nord and Ozkaya, 2012). Its operational and financial consequences are measurable and compound over a small number of years (Banker, Liang and Ramasubbu, 2021). In trading environments they are intensified by continuous availability requirements, vendor-dominated architectures and regulatory oversight (*FINMA Circular 2023/1 - Operational risks and resilience – banks*, 2024), making the absence of a quantitative prioritisation mechanism particularly costly (Ramasubbu and Kemerer, 2021; Haki *et al.*, 2023).

1.2 Problem Statement

The technical debt literature lacks a standardised, quantitative framework for measuring and prioritising debt at the service level in vendor-dominated enterprise IT. In a review of 44 primary studies on prioritisation, Lenarduzzi *et al.* (2021) found that quantitative multi-criteria approaches are uncommon and that empirical validation on industrial datasets is rare. Much of this gap follows

from where existing tools look. The dominant ones operationalise debt as the remediation effort implied by source code rule violations (Biazotto *et al.*, 2024), which makes them inapplicable where source code is unavailable, as is typical in trading IT. Even where code can be accessed, tools run against the same codebase return substantially divergent estimates (Amanatidis *et al.*, 2020), and they leave infrastructure lifecycle debt, platform obsolescence and vulnerability debt largely unaddressed. Operational data offers a route around this, though the link is not yet proven. DORA metrics, four delivery-performance measures from the DevOps Research and Assessment programme, are validated predictors of software delivery performance (Forsgren, Humble and Kim, 2018). The step from degraded operational indicators to elevated technical debt severity has not been established empirically, so this study treats it as a testable hypothesis. None of the literature reviewed applied a multi-criteria decision-making framework to live service management data to produce a ranked technical debt prioritisation instrument for financial trading IT. This specific absence motivates the framework developed here.

1.3 Research Aims and Objectives

This study develops a service-level quantitative framework for measuring and prioritising technical debt in mission-critical trading systems, grounded in operational service management data rather than source code analysis. Its objectives are as follows.

1. To review literature on technical debt measurement, prioritisation, financial enterprise IT governance and multi-criteria decision-making.
2. To identify and operationally define quantifiable technical debt indicators derived from live ITSM and CMDB records.
3. To design and apply a TOPSIS-based multi-criteria model for scoring and ranking services by technical debt priority.
4. To evaluate the framework analytically, by testing whether its rankings are stable under weight and temporal variation, reproduced by alternative methods, distinct from a single-criterion baseline and robust to the removal of individual services.

1.4 Research Questions

The study is guided by the following primary research question.

> RQ1: *How can technical debt in mission-critical trading systems be prioritised quantitatively at the service level using operational data?*

Two secondary questions support the investigation.

- RQ2: To what extent do operational service management metrics serve as valid proxies for technical debt severity in vendor-dominated enterprise IT environments?
- RQ3: How stable are the resulting service rankings when criterion weights are perturbed?

1.5 Significance of the Study

The main academic contribution is a working prioritisation framework that did not previously exist for this environment, connecting operational service management with how technical debt is studied in software engineering. To the best of current knowledge, no prior study has applied TOPSIS to live operational records from a financial trading institution for technical debt prioritisation. Practically, the framework operates on data routinely available in enterprise ITSM and CMDB systems without requiring source code access, making it directly applicable to vendor-dominated trading IT architectures and supporting auditable remediation decisions in regulated environments.

The framework measures debt indirectly through operational proxies, and expresses the result as a relative priority rather than an absolute quantity. The word "measuring" in the title carries that sense. It denotes a quantitative, reproducible ordering of services by debt priority, not a claim to have quantified the debt itself.

1.6 Thesis Structure

Chapter 2 reviews the foundations across four bodies of literature and identifies the research gap. Chapter 3 describes the research design, methodology and

ethical considerations. Chapter 4 presents the framework development and results, including data preparation and TOPSIS computation. Chapter 5 evaluates the framework against the criteria set in Chapter 3, covering stability, convergent and discriminant validity, rank reversal and weight-space robustness. Chapter 6 answers the research questions and gives a critical self-evaluation, the academic and practical contributions, and directions for future work.

2 Literature Review

Four bodies of literature intersect at the problem this thesis addresses: the theory of technical debt, approaches to its measurement and prioritisation, its governance in regulated financial IT, and multi-criteria decision-making. None of these streams alone is sufficient. Read together and critically, they expose a gap the proposed framework is designed to fill.

2.1 Scope and Rationale of the Review

Technical debt has become both a central concept and a persistent governance problem in software engineering and enterprise IT (Avgeriou *et al.*, 2023). Ward Cunningham introduced the metaphor in 1992, describing technical debt as the long-term cost of short-term technical compromises (Cunningham, 1992). Over time, the concept expanded from source code quality to encompass architectural, infrastructure, organisational and process dimensions of complex IT systems (Kruchten, Nord and Ozkaya, 2012).

Despite this conceptual broadening, the empirical and tool-supported literature remains concentrated at the code level. Static analysis tools and code-based metrics work tolerably well for greenfield development dominated by in-house code but are much less appropriate for enterprise environments where reliability depends on vendor platforms, legacy infrastructure and tightly integrated service stacks (Amanatidis *et al.*, 2020). In such environments, infrastructure, platform and vendor-related debt cannot be detected through code-centric tools.

What the literature does not yet offer is a quantitative, service-level mechanism for measuring and ranking technical debt where source code is unavailable. Yet in such contexts debt manifests in operational outcomes such as service instability, patch delay and infrastructure obsolescence (Ramasubbu and Kemerer, 2021;

Haki *et al.*, 2023). The following sections build a theoretical and methodological basis for the proposed framework.

2.2 Methodology for Literature Selection

The literature search was conducted between January and April 2026 using a systematic and reproducible approach, drawing on four principal academic databases: the University of Essex library, Scopus, IEEE Xplore and Google Scholar. Search terms are summarised in Table 2.1.

TABLE 2.1: Search terms used for literature review

Theme	Search String
TD foundations	`"technical debt" AND (metaphor OR taxonomy OR typology OR theory)`
TD measurement and tools	`"technical debt" AND (measurement OR "static analysis" OR SonarQube OR metric OR "code smell")`
TD prioritisation	`"technical debt" AND (priorit* OR "backlog management" OR remediation OR repayment)`
TD in enterprise and financial IT	`"technical debt" AND ("enterprise" OR "financial services" OR "trading" OR "legacy system" OR "regulated")`
TD automation	`"technical debt management" AND ("machine learning" OR intelligent OR automation)`
MCDM and TOPSIS	`TOPSIS AND ("multi-criteria" OR MCDM OR "SAW" OR "VIKOR" OR "decision making" OR priorit*)`
Patch management	`"patch management" AND (priorit* OR risk OR vulnerability OR SLA OR enterprise)`
Operational risk and DORA	`("DORA metrics" OR "mean time to recover") AND ("technical debt" OR "software delivery")`

Sources were screened against the pre-specified inclusion and exclusion criteria in Table 2.2.

TABLE 2.2: Inclusion and exclusion criteria

Criterion	Rule
Publication type	Peer-reviewed journal articles, conference proceedings or empirical technical reports
Language	English full text available
Date range	2019–2026, or foundational regardless of date
Thematic relevance	Directly addresses at least one of the four thematic pillars
Exclusion: scope	Opinion pieces without empirical grounding
Exclusion: topic	Agile process improvement without a measurable TD component
Exclusion: duplicates	Superseded by a more comprehensive journal version

Titles and abstracts were screened against these criteria first. Forward and backward citation snowballing on three anchor publications, Kruchten, Nord and Ozkaya (2012), Lenarduzzi *et al.* (2021) and Forsgren, Humble and Kim (2018), then surfaced relevant studies not captured by the database searches. This produced a combined pool of approximately 110 candidate sources, of which more than 50 were retained after full-text review for relevance and methodological quality.

The review uses narrative synthesis rather than a formal systematic review protocol, since the research spans three disciplines with differing methods and evidence standards. This approach supports the aim of building a coherent argument for a design science artefact (Hevner *et al.*, 2004).

2.3 Conceptual Foundations of Technical Debt

Before examining how technical debt is measured, it helps to see how the concept has evolved. Most tools reviewed in the next section are still calibrated to an earlier, narrower version of technical debt, and that mismatch is part of the problem this thesis addresses.

2.3.1 Origins and the Metaphor

Cunningham (1992) introduced the metaphor to explain a refactoring strategy to non-technical stakeholders. In his original formulation, shipping code that is "not quite right" resembles taking on financial debt, and each subsequent maintenance effort represents interest. If the principal is not repaid through refactoring, the cost of working with the system compounds until change becomes untenable.

The metaphor has proved productive but carries inherent limits. Unlike financial debt, the principal is an estimated remediation cost that changes as systems evolve, and the interest depends on uncertain future development activity. The proposed framework therefore does not rely on the metaphor for rigour. It grounds its estimates in service-level operational data, making interest costs concrete and observable.

Several influential practitioners refined the concept. McConnell (2008) introduced a cost structure distinguishing principal, recurring interest and compounding interest, and differentiated unintentional from intentional debt incurred as a conscious trade-off. Fowler (2009) proposed a quadrant distinguishing reckless from prudent and deliberate from inadvertent debt, though as a practitioner blog post its value lies in stakeholder communication rather than theoretical grounding. Kruchten, Nord and Ozkaya (2012) subsequently formalised a taxonomy covering code, design, architectural, test, build and documentation debt on the basis of a structured research programme, and this taxonomy underpins much of the subsequent empirical work.

2.3.2 Theoretical Development

Empirical studies have confirmed and extended this view. Besker (2020) reported that developers in industrial settings spend roughly 23% of their time dealing with the consequences of existing technical debt, with architectural debt exerting the strongest negative influence on productivity. Bedi and Kaur (2022) used regression analysis on open-source repositories and found that commit frequency, lines of code, code smells, and test coverage are significant predictors of technical debt, whereas developer reputation metrics are not.

Ahmad *et al.* (2026) approached the topic qualitatively, interviewing experts from four international companies. They identified five non-technical dimensions of debt accumulation in large-scale agile environments, namely social dynamics, process, people, documentation and requirements, with lack of communication emerging as the dominant driver. This reinforces the view of technical debt as a sociotechnical phenomenon that cannot be governed by technical measurement alone.

Vidoni, Codabux and Fard (2022) advanced a more radical argument through a game-theoretic lens, proposing that in complex systems technical debt cannot be completely removed and that its effects cannot be fully controlled even when individual items are repaid. The concept of "infinite technical debt" challenges the assumption that debt is always recoverable given sufficient effort. The argument has not been empirically validated in enterprise IT and is read here as a

theoretical provocation, though it implies that governance outputs should surface replacement decisions where incremental repayment is unlikely to be effective.

2.3.3 Expanding the Conceptual Perimeter

The conceptual perimeter of technical debt has progressively widened beyond source code to encompass systems engineering, regulatory lifecycle and vendor platform concerns. Kleinwaks, Batchelor and Bradley (2023) found in a systematic review that requirements, architecture, integration artefacts and verification artefacts can all accumulate debt, and that software-oriented approaches are poorly adapted to multidisciplinary systems engineering lifecycles.

Hanssen, Brataas and Martini (2019) identified an important mechanism, regulatory change as a debt trigger. In an open banking case study, architectural decisions that were adequate before the EU Payment Services Directive 2 (PSD2) became sources of scalability debt once new requirements were imposed without architectural review. The pattern is directly relevant to trading IT, where evolving compliance requirements can render existing architectures inadequate.

Yang *et al.* (2019) proposed a taxonomy of Commercial Off-the-Shelf (COTS) related technical debt covering seven key types, including vendor platform obsolescence and integration constraints. This broader view is essential for financial trading IT. Vendor platforms and COTS products often form the core of the operational stack (Haki *et al.* (2023)), and platform, infrastructure and vulnerability debt are invisible to standard static analysis. Managing them requires indicators from infrastructure records and operational monitoring rather than code repositories.

2.4 Measurement and Prioritisation Frameworks

Measurement is where the gap between concept and practice becomes most visible. The dominant tools operationalise technical debt as a function of source code. This works tolerably well in greenfield development but leaves infrastructure, platform and vulnerability debt largely invisible.

2.4.1 The Dominance of Static Analysis and Its Limits

Static code analysis tools such as SonarQube, SQALE and CAST dominate technical debt measurement, operationalising debt as estimated remediation effort derived from rule violations (Biazotto *et al.*, 2024). In a systematic mapping of 178 primary studies, Biazotto *et al.* (2024) found that most automated measurement artefacts remain code-level. They require substantial human intervention even at higher automation levels and rarely support end-to-end technical debt management. The focus on code leaves infrastructure-level, platform-level and vendor-related debt only weakly observed.

This limitation is material in enterprise environments where operational reliability depends on legacy infrastructure and third-party platforms rather than on in-house code alone. Ampatzoglou *et al.* (2022) presented SDK4ED as a comprehensive industry-oriented platform integrating several code-level techniques, reporting financial benefits over a three-year evaluation. Still, its support for infrastructure and vendor-platform debt remains limited. Tornhill and Borg (2022) confirmed the business relevance of code-level debt across 39 proprietary codebases, demonstrating that poor code quality was statistically associated with higher defect density, slower remediation and lower developer throughput. Neither, however, resolves the blind spots of a code-only perspective, which leaves legacy infrastructure and unsupported components unaddressed.

A further concern is measurement consistency. Amanatidis *et al.* (2020) evaluated agreement between three different technical debt assessment tools across 50 open source projects, finding that while the tools show strong overall statistical agreement, individual estimates and violation sets still vary considerably because each tool relies on its own ruleset and thresholds rather than an objective underlying quantity. Albuquerque *et al.* (2023) confirmed that infrastructure and process debt remain understudied relative to code and architectural debt. Static analysis is thus informative but insufficient for service-level prioritisation in vendor-dominated enterprise IT.

2.4.2 The Prioritisation Gap

Lenarduzzi *et al.* (2021) analysed 44 primary studies on technical debt prioritisation, finding that existing approaches cluster around cost-based, value-based and risk-based strategies, that most adopt a single primary criterion, that quantitative methods are less common than heuristic techniques, and that empirical validation on industrial datasets is rare. This conclusion directly motivates the quantitative contribution of the present work.

The operational stakes of inadequate prioritisation are concrete. de Toledo *et al.* (2021) showed in a production microservices system that targeted repayment of architectural technical debt produced an 84% reduction in incidents, grounding the case for systematic prioritisation in a measurable outcome rather than in theoretical argument. Practitioners consistently weigh both value and cost rather than a single strategy, and perceived value is highly context-dependent (Alfayez *et al.*, 2023). Pina, Seaman and Goldman (2022) found that prioritisation decisions reflect a combination of technical severity, business impact and available effort, with developers frequently torn between technically optimal choices and organisational constraints. Most IT managers nonetheless lack a formal technical debt management process, with cross-functional communication identified as the primary obstacle (Wiese and Borowa, 2023).

Stochel *et al.* (2023) proposed the Continuous Debt Valuation Approach (CoDVA), assigning a financial value to each debt item from remediation cost, interest rate and business risk exposure. Case studies show financial valuation can markedly change repayment priorities relative to effort-only approaches. CoDVA still relies on development-level data, however, and ignores operational service management signals such as incident rates or change failure rates. These are exactly the signals available where source code is inaccessible. They also establish that a quantitative framework must keep its value judgements transparent and auditable in regulated institutions.

2.5 Technical Debt in Financial and Regulated Enterprise IT

The limitations identified in the previous section are compounded in financial services. Vendor-managed architectures, regulatory obligations and continuous availability requirements create a context that the existing measurement literature was not designed for, and understanding that context is essential before proposing a framework intended to operate within it.

2.5.1 The Financial Services Context

The most directly relevant empirical grounding for the proposed framework comes from Haki *et al.* (2023), who conducted clinical research at Credit Suisse across more than 3,000 IT applications and defined a technical debt taxonomy covering code quality debt, infrastructure lifecycle debt, vulnerability debt and automation debt. Infrastructure lifecycle debt was measured as cumulative months of unsupported components, and vulnerability debt as weighted scanner-detected vulnerability counts. The taxonomy demonstrates that a major regulated institution treats infrastructure lifecycle and vulnerability debt as operationally significant categories and measures both through infrastructure records rather than source code, a precedent that directly supports the measurement approach taken in this thesis. Its empirical scope, however, covered only three development teams in one organisation and does not support generalisation.

Rolland and Lyytinen (2021) investigated architectural debt accumulation at a large Scandinavian financial institution, referred to in the study by the pseudonym BankAlpha, through eleven in-depth interviews. Two tensions emerged consistently: decentralised feature teams optimising for local throughput against the governance needed to manage debt at system level, and the pace of digital innovation against the long-term sustainability of the supporting architecture. Both are intensified by compliance obligations and competitive release pressure. The finding reinforces a pattern also visible in Haki *et al.* (2023), where organisational dynamics drive debt accumulation at least as much as technical decisions do, and management intent is the decisive governance variable in both cases.

2.5.2 Enterprise Systems and the Outsourcing Context

Banker, Liang and Ramasubbu (2021) examined 26 firms operating a COTS CRM system over its full eleven-year lifecycle. Using propensity score matching against firms using a non-customisable, web based CRM with zero technical debt, they found that a 10% increase in technical debt reduced gross return on assets, defined as gross profit scaled by beginning of year total assets, by 16% on average. Critically, this penalty more than doubled within six years of adoption as code decay compounded structural complexity. While all firms initially benefited from system customisation, those carrying median debt levels saw the net value of their system turn negative by year five, a pattern robust across alternative performance measures.

Ramasubbu and Kemerer (2021) extended this picture to 1,824 outsourced remediation projects, finding that client-vendor coordination only improved outcomes when both parties had agreed a long-term migration roadmap. Without this, increased coordination actually degraded performance. Rinta-Kahila *et al.* (2023) showed through system dynamics modelling how COTS customisation creates self-reinforcing debt accumulation loops that code-level tools cannot detect. Both findings support the governance argument of the present work. Ranking debt items must be embedded in a strategic remediation context, and governance-level visibility of infrastructure debt is required before those loops become irreversible.

2.5.3 DORA Metrics as Operational Debt Proxies

Forsgren, Humble and Kim (2018) drew on survey responses from over 2,000 unique organisations worldwide and showed that DORA metrics mean time to recover (MTTR), change failure rate (CFR), deployment frequency and lead time can distinguish high-performing from low-performing delivery teams, which makes these indicators usable where source code inspection is unavailable.

The inferential step this thesis takes goes beyond that validation. Using DORA metrics as proxies for technical debt severity requires the further claim that degraded operational performance reflects accumulated debt rather than inadequate staffing, immature release practices or organisational disruption. That

claim is plausible but not established as a general empirical law, so the framework treats it as a hypothesis rather than a premise.

Nord, Ozkaya and Woody (2021) provide the most direct grounding for this inferential step. Drawing on practitioner examples and software engineering literature, they classify delivery tempo degradation and deployment instability as observable technical debt symptoms that manifest in production records without requiring source code access. Critically, they identify elevated MTTR and CFR not as general performance shortfalls but as indicators that accumulated debt is impeding the system's ability to absorb and recover from change, positioning both as diagnostic signals of debt severity.

Paudel *et al.* (2025) found that technical debt density explained between 5% and 41% of lead time variance across six fintech components, with team structure and component ownership acting as confounding factors. This reinforces the case for multi-criteria measurement rather than reliance on any single proxy.

Nielsen and Skaarup (2022) offer complementary evidence from a vendor-managed public sector portfolio. Operational errors emerged shortly after TD-related resource constraints were imposed on a shared service component, and unsupported component versions surfaced only during infrastructure migration rather than routine monitoring. The public sector context limits transferability, but the mechanism matches the vendor-dominated profile of the present case.

The following table maps each operational indicator to the technical debt dimension. It is interpreted here as proxying, with the literature grounding each indicator's diagnostic value. The dimension assignments are this study's synthesis rather than claims made by the cited sources individually.

Table 2.3: Operational indicators mapped to technical debt dimensions

Indicator	TD Dimension	Literature Justification
Incident frequency	Architectural debt	de Toledo *et al.* (2021); Ramasubbu and Kemerer (2021); Ramasubbu, Kemerer and Woodard (2015); Nielsen and Skaarup (2022)
Mean Time to Recover (MTTR)	Infrastructure / platform debt	Forsgren, Humble and Kim (2018); Nord, Ozkaya and Woody (2021); Paudel *et al.* (2025)

Indicator	TD Dimension	Literature Justification
Change failure rate	Process debt	Forsgren, Humble and Kim (2018); Nord, Ozkaya and Woody (2021); Paudel *et al.* (2025)
Patch recency	Vulnerability debt	Tizio, Armellini and Massacci (2023); Markkanen and Frantti (2023)
Unsupported component months	Infrastructure lifecycle debt	Haki *et al.* (2023)

2.5.4 Security Debt and Patch Management

Security-related technical debt is most visible in the literature on patch management and vulnerability handling. Tizio, Armellini and Massacci (2023) analysed 86 Advanced Persistent Threats across 350 campaigns from 2008 to 2020, finding that enterprises delaying updates by one month faced 4.9 times higher odds of compromise, rising to 9.1 times at three months. Contrary to common assumption, most campaigns exploited publicly known vulnerabilities rather than zero-days, confirming that unpatched systems carry measurable risk. These multipliers are read as indicative of direction and magnitude rather than as calibrated benchmarks.

Markkanen and Frantti (2023) reviewed NIST guidance and the academic literature on patch management planning. They argue that uniform, time-based patching policies are increasingly inadequate given the shrinking window between disclosure and exploitation, and that patching should be treated as a risk-based business priority rather than a purely technical responsibility. Nurse (2025) framed patch prioritisation as a problem of maintaining SLA compliance, noting that business partners and clients drive patching decisions through supply chain agreements and contractual requirements. This aligns patch management directly with the service-level focus of this study.

Kula *et al.* (2019) studied dependency management at ING, showing that keeping upstream dependencies current is a persistent challenge in rapid-release banking environments and that delays contribute directly to technical debt. The study is based on a single organisation and may reflect ING-specific governance practices, so caution is warranted in applying its findings to trading IT environments with different release governance structures. Collectively, these studies establish that significant forms of technical debt in trading IT extend well beyond source code

and are detectable through ITSM and CMDB records rather than static analysis (Hanssen, Brataas and Martini, 2019; Ramasubbu and Kemerer, 2021; Haki *et al.*, 2023).

2.6 Multi-Criteria Decision-Making and TOPSIS

Prioritising technical debt involves balancing several criteria at once: remediation cost, operational risk, security exposure and regulatory compliance. Practitioners weigh these together rather than acting on a single factor (Alfayez *et al.*, 2023), which is the kind of problem multi-criteria decision-making (MCDM) methods are designed to solve.

Simple additive weighting (SAW) is the natural baseline, scoring each alternative as a weighted sum across the criteria. It is transparent but compensatory, letting a strong score on one criterion offset a weak score on another in proportion to their weights. A risk matrix, the other common baseline, is categorical and cannot finely separate a large service portfolio. TOPSIS is compensatory too, but scores each alternative by its distance to both an ideal best and an ideal worst rather than by a single weighted sum. This differs in aggregation structure, not guaranteed outcome, and Ciardiello and Genovese (2023) show that under some distance metrics TOPSIS reproduces SAW closely. Method choice is therefore treated here as a robustness question, examined in Chapter 5, rather than one of correctness.

2.6.1 TOPSIS: Capabilities and Limitations

Hwang and Yoon (1981) introduced TOPSIS, which ranks alternatives by their proximity to an ideal best and their distance from an ideal worst. The procedure builds a normalised, weighted decision matrix, identifies the ideal best and worst value per criterion, and ranks alternatives by a closeness coefficient, the ratio of the distance to the ideal worst to the sum of both distances (Albarak *et al.*, 2022). In a simulation comparing eight methods against a simple additive weighting benchmark, Zanakis *et al.* (1998) found TOPSIS the most robust to rank reversal of the methods that exhibit it, behind only the two that are reversal-free by construction.

A well-documented limitation is rank reversal, where adding or removing an alternative can reorder the others, because vector normalisation and the ideal solutions both depend on the current set (García-Cascales and Lamata, 2012). The same authors propose an absolute-mode correction that anchors normalisation and the ideal solutions to fixed bounds, making the ranking invariant to portfolio composition. This correction is implemented and evaluated in Section 5.5.

Three alternative methods warrant comparison. VIKOR identifies compromise solutions rather than a global ranking, fitting less well when a ranked remediation list is required (Shekhovtsov and Sałabun, 2020). AHP derives criterion weights through pairwise comparison and is frequently paired with TOPSIS, which supplies the ranking of alternatives (Ishak and Wanli, 2020). A weighted scoring matrix is simpler still but assumes comparable measurement scales, which fails when mixing continuous operational metrics with categorical compliance indicators. TOPSIS integrates normalisation, weighting and ranking in one procedure and handles mixed benefit and cost criteria directly, making it well suited to generating a ranked service portfolio.

Albarak *et al.* (2022) provide the most directly analogous precedent, combining TOPSIS with portfolio theory to prioritise technical debt and showing that it surfaces high-priority items likely to be deferred under effort-only approaches. TOPSIS has also been applied in other risk-intensive, regulated settings (Zytoon, 2020; Khan and Purohit, 2022). None operates on live service management data in a large financial enterprise, a defining characteristic of this study.

2.7 Transferability and Data-Quality Critique

Although the preceding sections provide strong motivation for the proposed framework, none of the existing applications reviewed operates on live operational records in large enterprise settings. Most rely on researcher-defined scenarios, synthetic datasets or controlled development artefacts. The proposed research differs by applying TOPSIS to operational records that are known to carry quality issues. Incident classifications vary between teams, recorded change failure rates reflect documentation practices as much as technical quality, and

vulnerability counts depend on scanner coverage. These limitations are acknowledged in Chapter 4, and the sensitivity of rankings to input variation is examined in Chapter 5 through weight perturbation, following Greco *et al.* (2019).

Using such data also raises ethical questions, since rankings derived from operational records may influence budgets, staffing and reputational assessments. These considerations are addressed in Section 3.2.

2.8 Synthesis

The four thematic sections together define the conceptual, methodological, contextual and technical landscape of this research.

2.8.1 Cross-Pillar Convergence

The four preceding sections converge on a common structural problem. The concept of technical debt has expanded to include infrastructure, vendor platforms and operational dependencies, but the dominant measurement and prioritisation methods have not kept pace (Yang *et al.*, 2019; Kleinwaks, Batchelor and Bradley, 2023; Biazotto *et al.*, 2024). The field now describes more forms of debt than its tools can reliably observe. Code-centric approaches remain useful but are poorly suited to environments whose most relevant liabilities sit in platform obsolescence, infrastructure lifecycle exposure or dependence on third-party systems (Haki *et al.*, 2023; Rinta-Kahila *et al.*, 2023).

Prioritisation shows the same lag. Technical debt is inherently multi-criteria, because cost, value, risk and operational impact must be considered together, yet the literature offers no validated quantitative method combining these dimensions into a transparent service-level ranking (Lenarduzzi *et al.*, 2021; Pina, Seaman and Goldman, 2022; Alfayez *et al.*, 2023). Meanwhile, unmanaged debt produces observable operational harm, in increased incidents, slower recovery, lower productivity and greater vulnerability exposure, and these effects are recorded in operational systems rather than code repositories (de Toledo *et al.*, 2021; Banker, Liang and Ramasubbu, 2021; Ramasubbu and Kemerer, 2021; Tornhill and Borg, 2022; Tizio, Armellini and Massacci, 2023).

Together these strands justify using operational indicators as auditable proxies for prioritisation rather than as direct measures of debt (Haki *et al.*, 2023; Tizio, Armellini and Massacci, 2023). This is the precise gap the framework developed in this thesis addresses.

2.8.2 Cross-Cutting Limitations

Several limitations qualify these conclusions. The empirical base for financial trading IT remains thin. The Credit Suisse, BankAlpha and ING studies are each restricted to single organisations (Kula *et al.*, 2019; Rolland and Lyytinen, 2021; Haki *et al.*, 2023), and whether their patterns generalise to securities trading environments remains open. This study contributes an operational dataset from such an environment, currently absent from the literature.

Existing TOPSIS applications to technical debt rely on controlled datasets, and the comparative work of Shekhovtsov and Sałabun (2020) shows that method choice matters less than weight specification when weights are uncertain. This underscores the importance of transparent and justified weight determination in the present study, which is addressed through a dual-configuration sensitivity analysis in Chapter 5. Behavioural and governance factors, including management intent, incentives and communication patterns, consistently exert a stronger influence on remediation outcomes than technical tools alone (Besker, 2020; Wiese and Borowa, 2023; Haki *et al.*, 2023). The scores the framework produces are a starting point for a governance conversation rather than a final answer. A service ranked high priority still requires human judgement about whether and how to act.

2.8.3 Literature Coverage and Thesis Contribution

Table 2.4 maps the reviewed literature to the thesis argument and makes the novelty explicit. No existing study combines service-level indicators, multi-criteria prioritisation, live operational data and a trading IT context into a single auditable framework.

TABLE 2.4: Overview of related work and research gap

Reference	Focus	SL Ind.	Multi-crit.	Fin. IT	Key gap
Kruchten, Nord and Ozkaya (2012); Fowler (2009); Cunningham (1992)	TD taxonomy	×	×	×	No measurement mechanism
Biazotto *et al.* (2024); Amanatidis *et al.* (2020); Ampatzoglou *et al.* (2022)	Static analysis tools	×	×	×	Inapplicable to vendor platforms
Lenarduzzi *et al.* (2021); Pina, Seaman and Goldman (2022); Alfayez *et al.* (2023)	TD prioritisation	×	Partial	×	Single criterion, no operational data
Stochel *et al.* (2023); Wiese and Borowa (2023)	Business-driven valuation	×	×	×	No service-level signals
Haki *et al.* (2023); Rolland and Lyytinen (2021)	TD in financial services	Partial	×	✓	No quantitative ranking output
Banker, Liang and Ramasubbu (2021); Ramasubbu and Kemerer (2021); Rinta-Kahila *et al.* (2023)	Enterprise COTS and outsourcing	×	×	✓	No prioritisation framework
Forsgren, Humble and Kim (2018); Nord, Ozkaya and Woody (2021); Paudel *et al.* (2025)	DORA metrics as TD proxies	✓	×	Partial	No ranking instrument
Tizio, Armellini and Massacci (2023); Markkanen and Frantti (2023); Nurse (2025)	Patch and vulnerability debt	Partial	×	Partial	Not linked to TD prioritisation
Albarak *et al.* (2022); Khan and Purohit (2022); Zytoon (2020)	TOPSIS / MCDM in IT	×	✓	×	No live data, no trading IT context
This thesis	**TOPSIS on ITSM/CMDB data**	✓	✓	✓	**None**

The research gap identified in the final row of Table 2.4 defines the problem that Chapter 3 translates into a concrete research design, and that Chapters 4 and 5 address through framework construction and evaluation.

2.8.4 The Research Gap

Related work does not offer a framework integrating service-level operational indicators, multi-criteria prioritisation and the constraints of regulated financial trading IT into a single auditable ranking instrument. Code-centric tools are structurally inapplicable to vendor-hosted platforms, and single-criterion models conflict with both practitioner behaviour and the multidimensional nature of operational risk. The framework developed here applies TOPSIS to five operational

indicators from live ITSM and CMDB records, producing a ranked instrument that operates without source code access. Its evaluation follows the logic of Greco *et al.* (2019) and Albarak *et al.* (2022): testing whether rankings hold when weights change, whether they are reproduced under alternative methods, and whether they diverge from a single-criterion baseline.

3 Research Design and Methodology

The research strategy and design decisions governing the framework's construction and evaluation proceed in two parts. The first section details the research design through to the evaluation framework, while the second addresses the ethical and professional considerations arising from analysing live operational data in a regulated financial institution.

3.1 Research Design

This study is grounded in a pragmatist research philosophy. Pragmatism holds that the value of a knowledge claim rests on whether the resulting artefact works in the context it was designed for (Kelly and Cordeiro, 2020). This suits the present study, because the aim is not to establish universal laws about technical debt but to build a quantitative framework useful for prioritisation decisions in a specific enterprise IT environment. Technical debt is treated here as a construct without a single measurement, since tools applied to the same codebase produce substantially different estimates (Amanatidis *et al.*, 2020). The framework therefore grounds its estimates in operationally defined proxies and treats the resulting scores as representations, not facts.

3.1.1 Research Strategy

Design Science Research (DSR) governs this study, as formalised by Hevner *et al.* (2004). DSR is appropriate because the primary output is an artefact rather than a theory. The artefact is a TOPSIS-based prioritisation framework operating

on live ITSM data, addressing the gap established in Chapter 2 and confirmed field-wide by Lenarduzzi *et al.* (2021).

Hevner *et al.* (2004) specify seven guidelines for rigorous DSR, mapped to this study in Table 3.1. The problem space, the absence of a service-level TD framework, connects to the solution space, the TOPSIS artefact, through input knowledge and research processes, positioning the framework as prescriptive design knowledge for regulated enterprise contexts.

Table 3.1: DSR guidelines and their implementation

DSR Guideline	Implementation in this Study
Design as an artefact	The TOPSIS-based prioritisation framework
Problem relevance	Established in Chapter 2, no existing framework integrates service-level operational metrics for TD prioritisation in trading IT
Design evaluation	Sensitivity analysis, convergent validation across alternative methods, discriminant baseline, rank-reversal test
Research contributions	A novel operational indicator set, and to the best of current knowledge the first application of TOPSIS to live ITSM and CMDB data for technical debt prioritisation in financial trading IT
Research rigour	TOPSIS applied per its formal specification (Section 3.1.4), with evaluation using established rank-correlation and robustness procedures (Section 3.1.5)
Design as a search process	Iterative refinement of indicator selection (literature candidates audited against ITSM/CMDB availability) and weight assignment
Communication of research	This dissertation, plus practitioner-accessible scoring output

The process follows the six-step model of Peffers *et al.* (2007): problem identification and motivation (Chapter 1 and Chapter 2), definition of objectives (Chapter 2), design and development of the artefact (Chapter 4), demonstration, evaluation (Chapter 5), and communication (Chapter 6). The sequence is not strictly linear, as indicator selection and evaluation criteria were refined iteratively when constraints in the available data emerged during development.

3.1.2 Case Study Design

The study is structured as a single-organisation embedded case study, situated within the trading IT division of a systemically important Swiss financial institution. Turnbull, Chugh and Luck (2021) define this design as an investigation into a single instance of observable complex phenomena, constrained by clearly identifiable boundaries. Discrete units of analysis, in this case individual IT services, are examined within that bounded organisational setting.

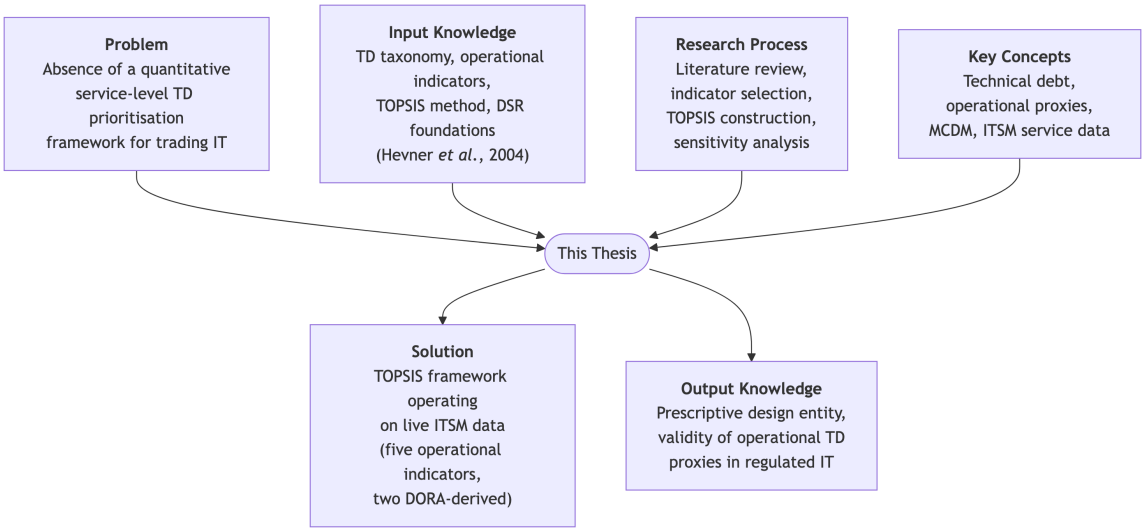

FIGURE 3.1: DSR Grid for the present study, adapted from Peffers *et al.* (2007)

Three factors make this design appropriate. First, the regulatory, operational and vendor-platform characteristics of Swiss financial IT resist generalisation from studies conducted elsewhere. Second, granular ITSM records are unavailable through surveys or experiments. Third, evaluating the artefact against operational data from its target environment satisfies the rigorous-evaluation requirement of Hevner *et al.* (2004).

The unit of analysis is the individual IT service, defined as a distinct configuration item in the organisation's ITSM instance. In this vendor-heavy environment, source-code access is unavailable for both outsourced components and on-premises vendor software (Ramasubbu and Kemerer, 2021). Service-level operational indicators are therefore tested as candidate proxies for technical debt prioritisation rather than assumed as established substitutes. The study is cross-sectional, analysing a fixed observation window rather than tracking change over time, consistent with the aim of demonstrating ranking validity at a point in time.

3.1.3 Data Sources and Collection

The literature review (Chapter 2) identified candidate indicators mapping technical debt dimensions to operational data (see Table 2.3). A data audit against available ITSM and CMDB fields refined this set prior to data collection. Documentation completeness was excluded because no structured ITSM or CMDB

field captures it consistently across services. The relevant records reside in unstructured repositories not amenable to uniform extraction. Test coverage ratio and architectural coupling were excluded because both require assets unavailable here, namely CI/CD build artefacts, source repositories or compiled code for dependency analysis (Amanatidis *et al.*, 2020), none of which is accessible for vendor-managed or outsourced components. Raw vulnerability density was excluded in favour of patch recency, though it too depends on CMDB scanner coverage and correct support dates, and supported components may still be vulnerable.

The five retained indicators are operationalisable from live records held in the organisation's ITSM instance and CMDB. Records were accessed under an agreed arrangement with the Head of Trading IT over a two-year observation window (January 2024 – December 2025) and anonymised at source, with service names replaced by alphanumeric identifiers and all team and vendor labels removed before analysis.

Table 3.2: Operational indicator definitions

Indicator	Operational Definition	Source	Unit	Direction
Incident frequency	Number of incidents per service per year.	ITSM	Incidents / year	Cost
MTTR	Arithmetic mean resolution time for incidents.	ITSM	Hours	Cost
CFR	Weighted incidents divided by the number of changes.	ITSM	Ratio	Cost
Patch recency	Share of service components running within a supported release window.	CMDB	Percentage (0–100)	Benefit
Unsupported component months	Total months of exposure to unsupported components in the service stack, divided by the service's component count (per-component intensity).	CMDB	Months / component	Cost

Two caveats apply. Incident frequency and MTTR are computed from all recorded incidents irrespective of severity, so both describe the overall incident profile. CFR is a derived proxy, calculated as weighted incidents divided by the number of changes, the formula the case organisation itself applies. It approximates the DORA change failure rate without the deployment-outcome labelling a true CFR requires. Because it shares its incident numerator with incident frequency, the

two criteria are not fully independent, a constraint whose effect on the ordering is bounded by the weight sensitivity analysis in Section 5.2.

Incident frequency, MTTR and CFR proxy the operational consequences of architectural, infrastructure/platform and process debt. Service instability and slow recovery are empirically linked to TD severity in vendor-managed portfolios (Ramasubbu and Kemerer, 2021; Nielsen and Skaarup, 2022). Change failure rate is included as a further operational signal, drawn from production change records without source code access. It is a recognised delivery-stability metric (Forsgren, Humble and Kim, 2018), and this study treats an elevated rate as one observable consequence of accumulated process and architectural debt, consistent with the operational framing adopted throughout.

The two CMDB-derived indicators, patch recency and unsupported component months, operationalise vulnerability and infrastructure lifecycle debt following Haki *et al.* (2023), differentiating services with similar current patch status but different historical accumulation. Throughout, operational indicators are treated as candidate proxies rather than established substitutes. The validity of this inference is the central empirical hypothesis tested in Chapter 5.

Unsupported component months needs one further normalisation before entering the decision matrix. In raw form it grows with the number of components a service comprises, so a large estate carrying a few unsupported components would outscore a compact service whose whole stack is out of support, making the indicator partly measure size rather than lifecycle debt (Ampatzoglou *et al.*, 2018). It is therefore converted to a size-relative intensity, $u_i = (\text{unsupported component months})_i / (\text{component count})_i$, a standard exposure-normalisation step for composite indicators whose sub-indicators differ in size dependence (Nardo *et al.*, 2005; Greco *et al.*, 2019). This is separate from and applied before the vector normalisation inside TOPSIS (Section 3.1.4). An intensity is preferred over a simple share of unsupported components because it preserves both how many components are affected and for how long. A fully supported service scores zero regardless of size, and a divide-by-zero guard covers services with no registered components, though none occurs in the dataset.

3.1.4 Framework Design

The analytical core is the TOPSIS method, formulated by Hwang and Yoon (1981) and validated through comprehensive reviews of its enterprise applications (Pandey, Komal and Dincer, 2023). TOPSIS addresses the single-criterion prioritisation limitation identified in Chapter 2 by ranking services according to their geometric proximity to ideal solutions after normalisation and weighting. Given a decision matrix with m services and n indicators, the method proceeds as follows.

The matrix is normalised using vector normalisation, which scales each value relative to the column norm to ensure comparability across indicators with different units and ranges (Pandey, Komal and Dincer, 2023). This normalisation runs down each criterion column, across the six services.

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^{m} x_{ij}^2}}$$

A weighted normalised matrix is constructed using criterion weight w_j.

$$v_{ij} = w_j \cdot r_{ij}$$

The positive and negative ideal solutions are identified.

$$A^+ = \{v_1^+, \dots, v_n^+\}, \quad A^- = \{v_1^-, \dots, v_n^-\}$$

Euclidean distances from each ideal are calculated.

$$S_i^+ = \sqrt{\sum_{j=1}^{n} (v_{ij} - v_j^+)^2}, \quad S_i^- = \sqrt{\sum_{j=1}^{n} (v_{ij} - v_j^-)^2}$$

The relative closeness coefficient C_i constitutes the final score.

$$C_i = \frac{S_i^-}{S_i^+ + S_i^-}, \quad C_i \in [0, 1]$$

A higher C_i indicates that a service is closer to the ideal best operational profile across the full indicator set and therefore has lower technical debt priority. Incident frequency, MTTR, CFR, and unsupported component months are cost criteria, where lower values are preferable, and patch recency is a benefit criterion, where higher is preferable.

The evaluation in Chapter 5 also draws on an absolute-mode variant of TOPSIS that is invariant to the set of services being ranked, following García-Cascales and Lamata (2012) and Yang (2020). It is used for the leave-one-out rank-reversal test and as the scorer for the weight-space analysis, where each sampled weighting is treated as a separate decision context. Two elements of the standard procedure are replaced. First, vector normalisation gives way to max normalisation against a fixed ceiling. Each criterion j is given a fixed range with a floor of zero and an upper bound u_j declared from organisational thresholds rather than from the sample, and each value is divided by that ceiling.

$$r_{ij}^{\text{abs}} = \frac{x_{ij}}{u_j}$$

Second, the ideal components are anchored to the extremes of each criterion scale rather than to the best and worst values observed among the services. Writing $v_{ij} = w_j\, r_{ij}^{\text{abs}}$ and respecting criterion direction, the positive and negative ideal components are fixed as

$$v_j^{+} = \begin{cases} w_j & \text{if } j \text{ is a benefit criterion} \\ 0 & \text{if } j \text{ is a cost criterion} \end{cases} \qquad v_j^{-} = \begin{cases} 0 & \text{if } j \text{ is a benefit criterion} \\ w_j & \text{if } j \text{ is a cost criterion} \end{cases}$$

Because u_j and both ideal components are fixed, they do not change when a service is added or removed, so the ranking is invariant to portfolio composition. The distances S_i^{+} and S_i^{-} and the closeness coefficient C_i follow exactly as in the standard procedure above. The declared bounds are listed in Appendix A.

Figure 3.2 illustrates the full pipeline from indicator collection through to ranked output.

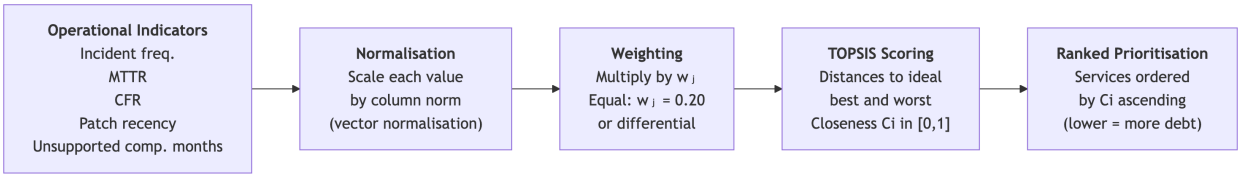

FIGURE 3.2: Framework pipeline: from operational indicators to ranked prioritisation

Two weight configurations are used. A baseline equal-weight configuration ($w_j = 0.20$) serves as the reference case. Equal weighting is the standard default in composite indicator construction in the absence of empirically justified differential weights (Greco *et al.*, 2019), and provides a transparent basis for the sensitivity analysis that follows. A secondary configuration assigns higher weight to MTTR and patch recency ($w_j = 0.30$ each) and lower weight to the remaining three indicators ($w_j = 0.1\overline{3}$ each), reflecting a practitioner judgement that service recovery time and delayed patching carry greater operational consequence in time-critical trading environments. This weighting is not derived from literature and is treated as an explicit assumption. Its influence on rankings is assessed in Chapter 5.

3.1.5 Evaluation Approach

No external ground-truth measure exists for the non-code debt this framework targets. Code-level tools disagree on the same codebase (Amanatidis *et al.*, 2020) and leave infrastructure, platform and vulnerability debt weakly observed (Albuquerque *et al.*, 2023; Biazotto *et al.*, 2024). The agreed governance scope also precluded interviewing service teams (Section 3.2). The framework is therefore evaluated analytically (Hevner *et al.*, 2004), establishing robustness, method-independence and discriminant value rather than criterion validity, which is left to future work.

Four strands are applied. The first tests ranking stability, extended beyond two discrete weight configurations by a Monte Carlo exploration of the full weight space (Greco *et al.*, 2019), with results finally read against the service profiles as a check of practical validity. The second tests convergent validity, re-ranking the same services with SAW and VIKOR and measuring each ranking's agreement with the TOPSIS order, a well-established comparison (Shekhovtsov

and Sałabun, 2020; Ciardiello and Genovese, 2023). Strong agreement shows that the order reflects the indicator profile, not the mechanics of one method. The third tests discriminant value against a single-criterion ranking by incident frequency. Divergence here shows the framework catches debt a single signal would miss, consistent with Albarak *et al.* (2022). The fourth tests structural rigour, removing services one at a time and checking whether the order of those remaining holds, which addresses the known rank-reversal effect in TOPSIS (García-Cascales and Lamata, 2012).

3.2 Ethical and Professional Considerations

The ethical considerations arising from this study concern data confidentiality and legal compliance, the responsible use of prioritisation outputs, and alignment with professional standards.

3.2.1 Data Confidentiality and Anonymisation

The data originate from a live Swiss financial institution that is not identified by name. All service, team, and vendor identifiers were replaced with alphanumeric codes before analysis. No employee data are present, since all five indicators are service-level aggregates that cannot be attributed to individuals. The data-sharing arrangement was established with the explicit authorisation of the Head of Trading IT. Only the data fields necessary for the analytical purposes of this study were exported, and no records were retained beyond the defined analytical period. All of these measures were applied before any data were transferred outside the organisation's infrastructure.

3.2.2 Legal Compliance

The organisation is subject to the revised Federal Act on Data Protection (nFADP), which entered into force on 1 September 2023 (*Federal Act on Data Protection*, 2023), enforced by the Federal Data Protection and Information Commissioner (FDPIC). Because the dataset contains no personal data, the nFADP's provisions on personal data processing do not apply directly. The principle of data minimisation, which the nFADP enshrines in alignment with international standards, was

nonetheless applied. Only the metrics required for the five TOPSIS indicators were extracted.

As a systemically significant institution, the organisation is also subject to FINMA Circular 2023/1 on operational risks and resilience, in force since 1 January 2024 (*FINMA Circular 2023/1 - Operational risks and resilience – banks*, 2024). The circular requires comprehensive ICT operational risk management, including incident monitoring and service resilience. The specific metrics used here are organisationally defined rather than prescribed by FINMA. No ethics committee approval was required under the University of Essex's research ethics framework, and a departmental no-objection confirmation was obtained.

3.2.3 Responsible Use of Prioritisation Outputs

A relative rank assigned to a service carries a risk of being misused to evaluate team performance. High-debt service profiles typically reflect historical design decisions or legacy constraints rather than the competence of the team currently managing the service (Ahmad *et al.*, 2026). This risk is not hypothetical in regulated financial institutions, where priority scores can influence budgets, staffing allocations and post-incident accountability.

The governance scope of this framework was agreed with the Head of Trading IT at the outset. The explicit understanding was that the framework would support a broader, portfolio-level view of service health and inform governance conversations, rather than serve as a definitive or binding prioritisation instrument. The closeness coefficient is a decision-support signal derived from operational proxies, and it was presented as such from the start of the engagement.

Team leads were not involved in the creation of this framework, as the study was conducted solely by the author as part of a master's thesis. No outputs were shared with service teams during the research period. Any operational use of the priority scores beyond the agreed portfolio management scope would require a separate governance decision by the Head of Trading IT before deployment.

3.2.4 Professional Standards and Limitations

This study followed the BCS Code of Conduct (*Code of Conduct for BCS Members*, 2022), including the obligations to act with integrity, respect the rights of third parties and ensure the artefact is fit for purpose. Three limitations clarify scope. The single-site design means indicator weights and TOPSIS configuration are calibrated for one organisation and should not be transferred without revalidation. The cross-sectional window means the framework captures a snapshot, so recently remediated services may be under-ranked. Finally, the five indicators do not exhaust all dimensions of operational technical debt. Indicators for documentation, test coverage or architectural coupling were excluded on data availability and remain candidates for future iterations.

4 Framework Development and Results

Two years of live ITSM and CMDB records, from January 2024 to December 2025, at a regulated Swiss trading institution supply the raw material for this chapter. The sections below document and justify the indicator values extracted from those records, the normalisation and weighting decisions applied to them, and the resulting TOPSIS scores.

4.1 Data Preparation

The trading IT portfolio includes 108 services. Fifty-three are small vendor applications such as vendor graphical user interfaces, and were excluded because they require no significant ongoing activity from the portfolio's IT teams.

From the remaining services, six were selected for demonstration on the basis of data completeness. Each holds a continuous record across all five indicators for the full two-year window in both systems, is registered as an active configuration item and has been in operation for more than thirty-six months. Service identifiers were replaced with alphanumeric codes before analysis (Section 3.2), and the dataset contains no team, vendor or individual identifiers. Two consequences follow from such a small demonstration set. First, the services selected tend to carry higher operational activity, so absolute indicator values are not representative of the full portfolio. Second, a six-service decision matrix limits the discriminatory power of the rank-stability tests (Section 5.9). The contribution is therefore the working instrument and its evaluation logic, not a portfolio-wide ranking, which is identified as future work once indicator coverage across the portfolio is complete.

4.1.1 Indicator Extraction and Quality Checks

The five indicators defined in Section 3.1.3 were extracted from the organisation's ITSM instance and CMDB on 11 June 2026. Only production records were included in the extraction.

Prior to normalisation, several quality filters were applied to the raw incident data. Incidents sharing a common root cause but recorded as separate tickets, a pattern typical of alarm storms, were consolidated into a single incident to avoid inflating frequency counts. Incidents with an MTTR below 0.1 hours were excluded, since resolutions at that speed most likely reflect automatic closure after a false alarm rather than genuine remediation effort. A 98th percentile cap addressed the upper tail of the MTTR distribution, excluding incidents whose resolution times more plausibly reflect administrative inactivity than active resolution effort.

All CMDB records were found to be complete. No imputation or worst-case substitution was required. The cumulative unsupported-component-months figure extracted from the CMDB was divided by each service's registered component count to yield the per-component intensity defined in Section 3.1.3, with numerator and denominator drawn from the same extract.

4.1.2 Data Limitations

The data carry inherent quality limitations that must be acknowledged before interpreting the results. Because the organisation does not document CFR directly, it is derived from incident and change records as described in Section 3.1.3. Incident frequency, MTTR and CFR all depend on the accuracy and completeness of the incidents opened and closed. Patch recency is bounded by CMDB coverage and scanner configuration, so services with incomplete records may appear more current than they are. Because unsupported component months is expressed per component, the accuracy of the CMDB component count becomes a dependency of the indicator. A mis-scoped denominator would bias a service's lifecycle-debt intensity, though component records were complete for the six demonstration services.

Recording practices also vary across teams. Some contacts arrive by phone, email or chat and become a formal ITSM record only at the receiving team's discretion.

Others are handled as defects in a separate issue tracker and never escalated. Services where informal handling is common may therefore record lower incident counts and CFR values than their true failure rate warrants, compressing their priority scores. MTTR is exposed at closure, since a team that leaves resolved incidents open will show a longer mean resolution time than its actual recovery warrants. Inconsistent severity labelling has limited influence here, as incident frequency and MTTR are computed across all severities.

These limitations do not invalidate the proxy approach. Scores should be read as ordinal priority signals rather than exact measurements, and the sensitivity analysis in Chapter 5 tests whether the ranking is robust to the input variation they may introduce.

4.1.3 Operational Indicator Values

Table 4.1 presents the extracted indicator values for all six services. Their definitions are given in Section 3.1.3.

Table 4.1: Indicator values per service, 2025

Service	Incident Freq. (per year)	MTTR (hrs)	CFR	Patch Recency	Unsup. Comp. Mo. / comp.
SVC-01	64	45.35	0.83	53.10	5.67
SVC-02	30	25.26	0.19	61.90	2.42
SVC-03	21	13.31	0.38	100.00	0.00
SVC-04	574	41.91	3.38	100.00	0.00
SVC-05	171	23.72	1.52	100.00	0.00
SVC-06	302	13.11	3.96	52.94	18.88

The exact content of the files that have been used for the TOPSIS calculation can be found in Appendix B.

4.2 Indicator Normalisation and Weighting

Each indicator was normalised using vector normalisation, as specified in Section 3.1.4, which removes differences in scale and unit while preserving proportional relationships between observations. Min-max normalisation was considered but not adopted, since it rescales values to fixed bounds and can compress the distribution of extreme observations, altering the relative distances between

alternatives on which distance-based methods such as TOPSIS rely. This choice is discussed further in relation to rank reversal in Section 5.5, where the normalisation method is identified as one of two structural sources of rank instability in TOPSIS (García-Cascales and Lamata, 2012).

Two weight configurations are applied, as specified in Section 3.1.4. The baseline equal-weight configuration assigns $w_j = 0.20$ to each of the five criteria and serves as the reference case. The differential configuration assigns higher weights to MTTR and patch recency, consistent with the design rationale established in Chapter 3. The differential weights are $w_{\text{MTTR}} = w_{\text{Patch Recency}} = 0.30$ and $w_{\text{IncFreq}} = w_{\text{CFR}} = w_{\text{UnsupComp}} = 0.1\overline{3}$.

TABLE 4.2: Weight configurations

Configuration	Incident Freq.	MTTR	CFR	Patch Recency	Unsup. Comp. Months
Equal (baseline)	0.20	0.20	0.20	0.20	0.20
Differential	$0.1\overline{3}$	0.30	$0.1\overline{3}$	0.30	$0.1\overline{3}$

4.3 TOPSIS Computation and Rankings

The computation follows the TOPSIS formulation specified in Section 3.1.4 and was executed using the Python artefact in Appendix A. This process calculates the closeness coefficient C_i for each service based on its Euclidean distance from the ideal solution. A higher C_i indicates that a service is closer to the positive ideal solution, while a lower C_i signals higher technical debt priority. Since the framework is intended to guide prioritisation, the service with the least favourable score receives the highest rank.

Table 4.3 presents the full results under both weight configurations. The table is sorted from highest to lowest debt priority, so rank 1 equals highest priority for remediation.

TABLE 4.3: TOPSIS results, 2025 (higher C_i, lower debt priority)

Service	C_i (Equal)	Rank (Equal)	C_i (Adjusted)	Rank (Adjusted)
SVC-06	0.3204	1	0.4415	1
SVC-04	0.4778	2	0.4512	2
SVC-01	0.6619	3	0.4976	3
SVC-05	0.7790	4	0.7615	5

Service	C_i (Equal)	Rank (Equal)	C_i (Adjusted)	Rank (Adjusted)
SVC-02	0.8297	5	0.7175	4
SVC-03	0.9774	6	0.9809	6

To assess temporal stability, the framework was applied to the 2024 dataset using the same weight configurations. Table 4.4 presents the resulting closeness coefficients and rankings for both configurations.

TABLE 4.4: TOPSIS results, 2024 (higher C_i, lower debt priority)

Service	C_i (Equal)	Rank (Equal)	C_i (Adjusted)	Rank (Adjusted)
SVC-04	0.4837	1	0.5038	1
SVC-06	0.5163	2	0.6048	3
SVC-02	0.6999	3	0.5102	2
SVC-05	0.7058	4	0.7092	5
SVC-01	0.7759	5	0.6231	4
SVC-03	0.8492	6	0.8816	6

The dominant lifecycle-debt signal sits with SVC-06 in both years. Its per-component exposure is the highest in the portfolio (16.41 months per component in 2024, rising to 18.88 in 2025), and by 2025 this combines with the worst change failure rate (3.96) and the worst patch recency (52.94) to place it first for remediation under both weight configurations. SVC-01's exposure also rose year on year, from 0.24 to 5.67 months per component, but spread across a large component base it remains a second-order signal; together with the portfolio's highest MTTR (45.35 hours) and second-worst patch recency (53.10) it places SVC-01 third in 2025. This illustrates the effect of the per-component normalisation directly. Under a cumulative measure, SVC-01's large estate would have produced the biggest raw figure and drawn it to the top of the ranking, whereas the size-adjusted intensity correctly subordinates it to SVC-06, a compact service about half out of support, and for far longer. These movements are drawn from the same complete CMDB extract that underlies the indicator (Appendix B).

The closeness coefficients and rankings produced under both weight configurations and both years constitute the analytical output of the framework and the empirical basis for Chapter 5, which tests their stability, method-independence,

discriminant value and structural rigour before interpreting the resulting priorities.

5 Evaluation and Discussion

The framework developed in Chapter 4 ranks six production services from five operational indicators held in the organisation's ITSM instance and CMDB. Before the order is interpreted, it has to earn trust. This chapter therefore tests it from several directions: whether it is stable under varying weights and observation years, whether alternative methods reproduce it, whether it adds information over a single-criterion view, and whether it survives the removal of individual services.

5.1 Evaluation Criteria

The framework is evaluated against the criteria set out in Section 3.1.5, forming an analytical evaluation in the sense of Hevner *et al.* (2004) Guideline 3. Five properties are at issue. Ranking stability asks whether the order survives reasonable weight variation and holds across observation periods, assessed throughout with Kendall's tau (τ). Convergent validity asks whether alternative methods reproduce the ordering, and discriminant value whether the ranking adds anything over a single-criterion incident-frequency baseline. Structural rigour requires the remaining order to hold when any service is removed. Practical validity, finally, asks whether the rankings can be read sensibly against the service profiles they describe.

5.2 Ranking Stability

Kendall's tau measures the similarity between two rankings and is well established in Multi-Criteria Decision-Analysis (MCDA) evaluation (Shekhovtsov, 2021). It scales concordant minus discordant pairs to $[-1, 1]$, where $\tau = 1$ is perfect agreement. With only six services the p-value is obtained exactly across all

6! = 720 orderings. The small sample limits the test's power, but the limitation reflects portfolio size rather than weak agreement, since p-values fall as the sample grows when a real effect is present (Chén *et al.*, 2023). The p-values in Table 5.1 are therefore read as effect sizes rather than accept-or-reject decisions. All figures refer to the standard formulation. The absolute-mode variant is examined in Section 5.6.

TABLE 5.1: Kendall's τ across ranking comparisons, standard TOPSIS

Comparison	τ	p
Weight sensitivity 2025 (equal vs adjusted)	0.867	0.017
Weight sensitivity 2024 (equal vs adjusted)	0.733	0.056
Temporal stability equal weights (2025 vs 2024)	0.467	0.272
Temporal stability adjusted weights (2025 vs 2024)	0.600	0.136

Weight sensitivity is high in both years ($\tau = 0.867$ in 2025, 0.733 in 2024). Reweighting MTTR and patch recency leaves the priority order largely intact, moving only adjacent mid-field services. Temporal stability is moderate ($\tau = 0.467$ equal, 0.600 adjusted), and the movement follows real profile change. SVC-06 is the clearest anchor. It carries the portfolio's heaviest per-component lifecycle exposure in both years. Ranked second in 2024, it rises to the top priority in 2025 as its change failure rate and patch position deteriorate. SVC-01 climbs from fifth in 2024 to third in 2025 as its patch recency collapses and its per-component unsupported exposure rises. Cross-year movement is reported as descriptive context, not as a quality criterion in its own right. Services evolve between periods, and a faithful ranking should follow that change. In every run the lowest-priority position is identical, and the same three services occupy the high-priority half in 2025 under both configurations.

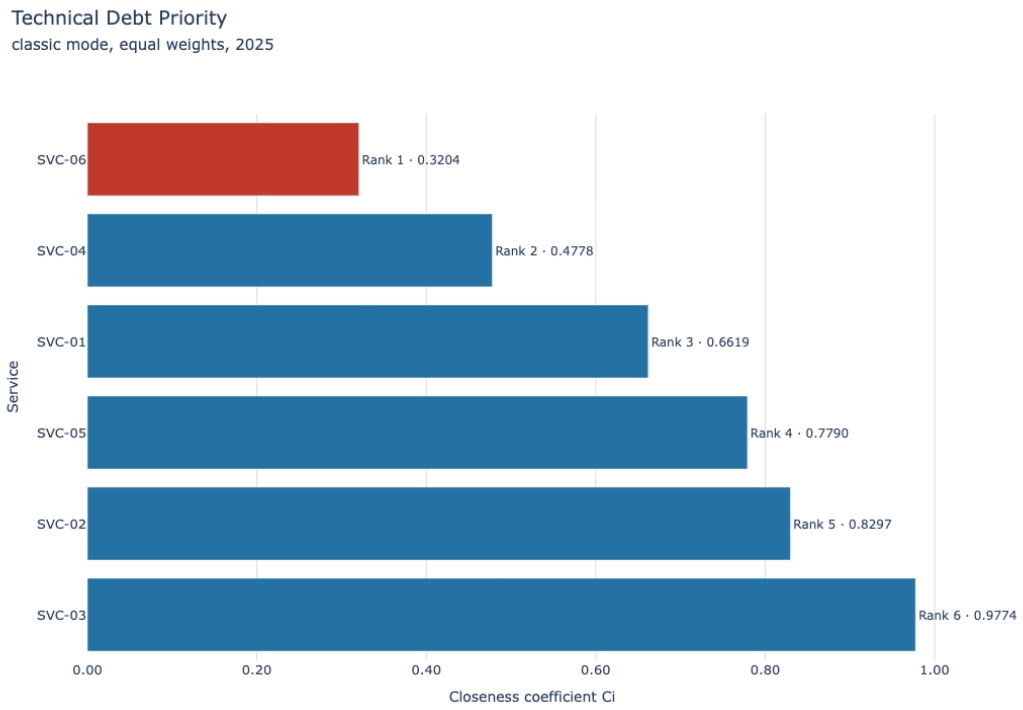

FIGURE 5.1: Technical debt priority ranking for 2025, equal weights.

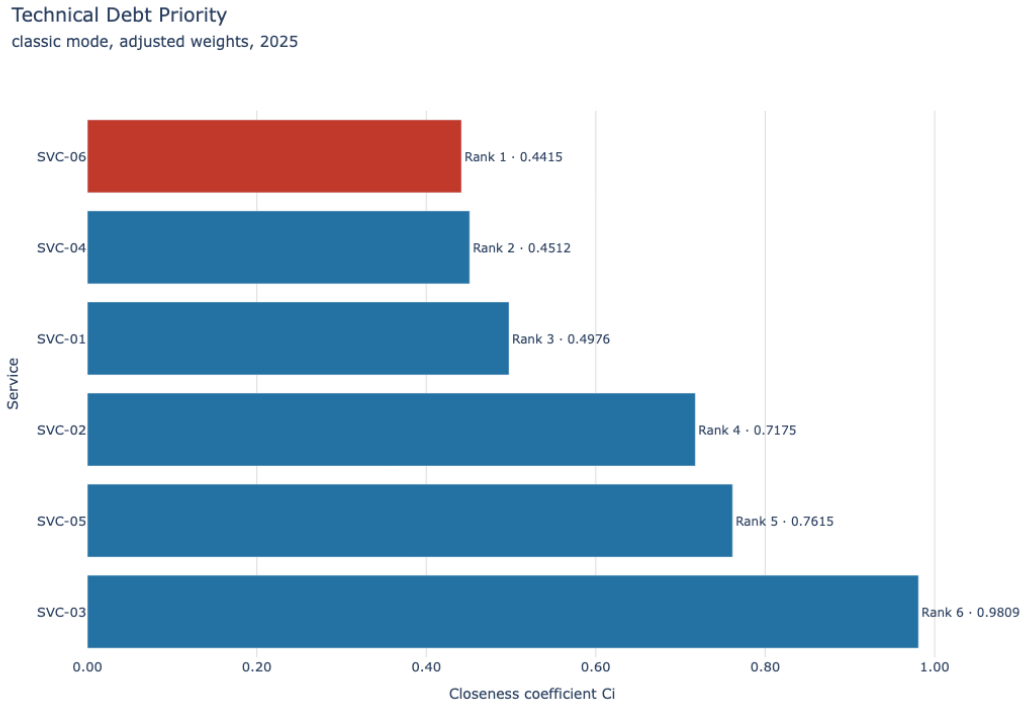

FIGURE 5.2: Technical debt priority ranking for 2025, adjusted weights.

5.3 Convergent Validity Across Methods

To test whether the ordering depends on the choice of method, the six services were re-ranked with SAW and VIKOR and compared against standard and absolute TOPSIS. SAW used Weitendorf (min-max) normalisation to avoid the division by zero that classic cost normalisation would produce on the dataset's zero values (Thien, Dung and Trung, 2024). VIKOR followed the standard S, R and Q procedure with $v = 0.5$. Full figures are in Appendix C.

Re-ranking reproduces the TOPSIS order closely. In 2025 under equal weights the absolute-mode ranking agrees perfectly with both alternatives ($\tau = 1.000$), and the standard formulation agrees strongly ($\tau = 0.867$ against each), all four methods placing SVC-06 first and SVC-03 last. Agreement remains strong under adjusted weights in 2025 (absolute $\tau = 0.867$; classic $\tau = 0.733$). In 2024 the methods again concur on the high-priority pair, SVC-04 and SVC-06, and on SVC-03 as the invariant lowest. However, they disagree on the internal order of the closely scored middle, and under adjusted weights on which of SVC-04 and SVC-02 leads. Agreement is therefore lower (absolute τ between 0.333 and 0.733). The reason is the aggregation structure. SVC-06 carries the portfolio's worst change failure rate, worst patch recency and heaviest per-component lifecycle exposure, weaknesses that VIKOR's regret term and SAW's linear sum weight heavily, while SVC-04 carries by far the highest incident count, whose magnitude vector-normalised distance preserves and the other two methods compress. As before, the disagreement concentrates on services separated by small score margins, while the broad prioritisation, the identity of the top cluster and the bottom anchor, is method-independent, a point revisited in Section 5.9.

5.4 Discriminant Value Against a Single-Criterion Baseline

A multi-criteria framework only earns its complexity if it says something an obvious single signal would not. Tested against a ranking by incident frequency alone, the composite ordering diverges ($\tau = 0.733$ in 2025, 0.200 in 2024, with the full comparison in Appendix C). The divergence concentrates in two services.

SVC-01 rises from fourth on incident count to third overall, carrying the portfolio's highest MTTR, a patch recency of 53.10 and a per-component unsupported exposure of 5.67 that an incident count does not register. SVC-05 falls from third to fourth for the mirror-image reason, as its incident volume is offset by full patch recency and no unsupported components. In 2025 the divergence is modest, because the top of the ranking aligns partly with incident frequency. SVC-06 carries both a high incident count (302) and the worst structural profile. The ordering is still not reducible to a single signal, and the divergence is pronounced in 2024 ($\tau = 0.200$). The result is contingent: indicators that moved together would have reproduced the incident ranking however many were added. They carry distinct information instead, which is the condition under which the multi-criteria design adds value (Albarak *et al.*, 2022). What the comparison does not establish is that the composite ordering is the more correct one. That rests on the indicator-to-dimension mapping inherited from the literature (Table 2.3).

5.5 Structural Rigour Under Service Removal

Structural rigour was examined by removing each service in turn and re-ranking the remainder. Under the standard formulation the 2025 order is preserved under all six removals (Table 5.2). The 2024 ranking, by contrast, reorders under three removals, of SVC-03, SVC-04 and SVC-06 (Appendix C). The lowest-priority service never moves in either year, but in 2024 the identity of the top-priority service depends on which others are present. This is the limitation of set-relative distance aggregation identified by García-Cascales and Lamata (2012), confined here to the more compressed 2024 profiles rather than the top of the 2025 ranking.

Table 5.2: Leave-one-out rank changes, standard TOPSIS, 2025 equal weights

Service removed	Outcome
SVC-01	Order preserved
SVC-02	Order preserved
SVC-03	Order preserved
SVC-04	Order preserved
SVC-05	Order preserved
SVC-06	Order preserved

To remove the effect, the absolute-mode correction of García-Cascales and Lamata (2012), specified in Section 3.1.4, was implemented. Vector normalisation is replaced by normalisation against fixed criterion bounds, and the ideal solutions are anchored direction-aware to fictitious alternatives at the extremes of each criterion scale, addressing the cost-attribute limitation that Yang (2020) identifies in the original formulation. Under this variant the order is preserved across all six removals in both years (Appendix C), guaranteed rather than incidental. Yang (2020) proves that variants with set-independent bounds and ideals are ranking stable by construction. The gain carries a cost, since the fixed bounds must be justified from organisational thresholds rather than sample statistics (values in Appendix A). Anchoring the ranking to a fixed frame, however, changes more than its stability properties.

5.6 Behaviour of the Absolute-Mode Variant

The absolute-mode variant tracks the standard formulation closely in 2025. Its order (SVC-06 $\succ$ SVC-04 $\succ$ SVC-01 $\succ$ SVC-02 $\succ$ SVC-05 $\succ$ SVC-03) differs only in the adjacent SVC-02 and SVC-05 pair, and it agrees with SAW and VIKOR at $\tau = 1.000$ under equal weights. Its distinctive behaviour appears in 2024, where fixing the reference frame reorders the closely scored middle. Under adjusted weights it leads with SVC-02 where the standard formulation leads with SVC-04, because a service that is merely worst-in-sample on a criterion no longer counts as extreme unless it approaches the declared scale limit. The reason is the fixed reference frame introduced in Section 5.5.

The variant's year-on-year concordance ($\tau = 0.733$ equal, 0.333 adjusted) follows from the same genuine profile movements and, per Section 5.2, is not read as a deficiency. The absolute frame retains two properties no other method offers here. Its rankings survive every leave-one-out removal unchanged in both years, and its scores remain comparable across periods, because the other methods rescale against whichever services are in that year's sample.

The practical consequence is a division of labour. The standard formulation, triangulated against SAW and VIKOR, remains the primary signal. It is decisive in 2025, where all four scorers agree SVC-06 is the top priority, and the 2024 result

shows why triangulation is reported rather than a single number. The absolute-mode variant additionally contributes rank-reversal immunity and cross-period comparability, certifying that the primary ranking is not an artefact of sample composition.

5.7 Weight-Space Robustness

Because two discrete configurations cannot capture the plurality of defensible judgements a weighting encodes (Greco *et al.*, 2019), a Monte Carlo procedure sampled 20,000 weight vectors uniformly from the Dirichlet simplex over the five criteria and recomputed the ranking for each. The absolute-mode scorer is used because each sampled weighting is a distinct decision context, and its set-invariant reference frame (Section 5.6) keeps rank-reversal effects out of the weight-space result. This approximates, in tractable form, the stochastic acceptability analysis those authors advocate. The run is reproducible from a fixed seed (50), and repeating it with a different seed changed no reported probability by more than 0.011.

Table 5.3 reports the rank-acceptability distribution for 2025. The bottom of the ranking is close to weight-independent, with SVC-03 at the lowest priority in 96.4% of weightings. The top is concentrated rather than evenly split. Rank 1 falls to SVC-06 in 65.1% of draws, with SVC-04 (26.2%) and SVC-01 (8.8%) the only other services ever reaching it. The extreme low priority is thus robust to any weighting, while the single top service is dominated by SVC-06 but weight-dependent within a stable three-service cluster, consistent with Section 5.6.

Table 5.3: Rank-acceptability distribution, absolute-mode TOPSIS, 2025 (20,000 weight samples, seed 50), cells are rank probabilities

Service	R1	R2	R3	R4	R5	R6
SVC-01	0.0876	0.3231	0.3708	0.2186	0.0000	0.0000
SVC-02	0.0000	0.0002	0.1974	0.3209	0.4456	0.0358
SVC-03	0.0000	0.0000	0.0000	0.0000	0.0358	0.9642
SVC-04	0.2616	0.4218	0.1426	0.1740	0.0000	0.0000
SVC-05	0.0000	0.0000	0.2202	0.2780	0.5018	0.0000
SVC-06	0.6508	0.2550	0.0690	0.0086	0.0168	0.0000

Repeating the analysis on 2024 data yields a flatter distribution (Table C.11). SVC-03 occupies the lowest rank in 72.0% of weightings rather than 96.4%, and the top rank spreads across three services, with SVC-04 taking it in 42.6% of draws, SVC-06 in 32.3% and SVC-02 in 25.1%. This mirrors the weaker 2024 figures in Section 5.2 and has the same cause, the more compressed 2024 profiles, confirming that weight sensitivity here is a property of the data rather than of the method.

5.8 Practical Validity

Three services illustrate what the rankings capture. SVC-06 holds rank 1 under both configurations and both TOPSIS modes, pairing the portfolio's heaviest per-component lifecycle exposure (18.88 months per component) with the worst change failure rate (3.96), the worst patch recency (52.94) and the second-highest incident count (302 per year). Table 2.3 places this debt across the infrastructure lifecycle, vulnerability, process and architectural dimensions (Nord, Ozkaya and Woody, 2021; Haki *et al.*, 2023); SVC-06 is worst or near-worst on four of the five indicators, which is why it is robustly top. SVC-01 holds rank 3 and is the multi-criteria exemplar: a modest incident count (64 per year, fourth of six) masks the portfolio's highest MTTR (45.35 hours), its second-worst patch recency (53.10) and its second-worst per-component exposure (5.67), the signature of infrastructure and vulnerability debt behind a quiet incident profile. SVC-04 holds rank 2 with the mirror-image profile: the highest incident frequency (574 per year), an MTTR of 41.91 hours and a CFR of 3.38, yet no unsupported components and full patch recency, the indicators of architectural and process debt (de Toledo *et al.*, 2021; Nord, Ozkaya and Woody, 2021). SVC-03 has the lowest priority in every run and is the reference case for a well-maintained service, with the fewest incidents, low MTTR and CFR, full patch recency and no unsupported components. The one point where the methods disagree, the 2024 order within the closely scored high-priority services, is examined in Section 5.6.

A cross-sectional design cannot establish whether the observed metrics are downstream effects of technical debt severity or arise from other operational factors, so proxy validity is assessed on the basis of convergence with theory rather than

demonstrated causal mechanism. Each profile corresponds to a debt pattern the literature documents (Ramasubbu and Kemerer, 2021; Nord, Ozkaya and Woody, 2021; Haki *et al.*, 2023), and the profiles are distinct from one another rather than expressions of a single underlying deficiency, which is what makes the ordering readable against Table 2.3. That correspondence is a consistency check rather than an independent test, since the ranking is computed from the same indicators that constitute each profile.

5.9 Limitations of the Evaluation

Every strand draws on the same dataset and indicator set used to build the framework, so the triangulation and baseline comparisons establish convergent and discriminant validity but not criterion validity, for which no independent measure of the non-code debt in scope exists. External validation is the main avenue for future work. The six-service portfolio also limits the power of the Kendall's tau tests, since $\tau = 0.600$ cannot reach conventional significance across only 720 permutations even where agreement is substantively meaningful. The evaluation also depends on the accuracy of the per-component denominator introduced by the size-normalisation of the unsupported-components indicator: the ratio is only as reliable as the CMDB's component count for each service. The method disagreement in Section 5.3 and Section 5.6 adds a further boundary. The single top rank is dominated by SVC-06 but remains weight-dependent within the SVC-06, SVC-04 and SVC-01 cluster across the weight space, turning on whether concentrated structural exposure is judged more urgent than diffuse operational debt, a compensability judgement the framework informs rather than settles. Finally, the cross-sectional design captures a single snapshot, so a service mid-remediation may look worse than warranted. Longitudinal evaluation is flagged in Chapter 6, and the data quality limitations in Section 4.1.2 apply here, with middle-position rankings warranting careful reading.

6 Conclusion

The three research questions posed at the outset each have a direct answer. This chapter states those answers, reflects on their limits, and identifies what future work needs to address.

6.1 Research Summary

This study set out to close a specific gap. Vendor-dominated trading IT lacked a quantitative, service-level instrument for prioritising technical debt, since code-centric tools cannot operate where source code is inaccessible. Following a Design Science approach (Hevner *et al.*, 2004), five operational indicators were derived from the literature, mapped to technical debt dimensions (Table 2.3), then operationalised from two years of live ITSM and CMDB records at a regulated Swiss trading institution. A TOPSIS model ranked six production services under two weight configurations and two observation years. The evaluation in Chapter 5 tested the ranking from five directions, from weight sensitivity across the entire weight space to structural rigour under service removal. The artefact works in the sense the pragmatist stance of Chapter 3 demands, producing a defensible, reproducible ordering from data the organisation already holds.

6.2 Findings Against Research Questions

RQ1 asked how technical debt in mission-critical trading systems can be prioritised quantitatively at the service level using operational data. The framework itself is the answer. Five indicators, three from ITSM records and two from the CMDB, feed a TOPSIS model that orders services by debt priority without any recourse to source code. The multi-criteria design earns its place. The discriminant test showed that the ordering diverges from an incident-only view ($\tau = 0.733$

in 2025, and more sharply, 0.200, in 2024). That divergence was not guaranteed. Indicators that moved together would have reproduced the incident ranking and made TOPSIS redundant. SVC-01 rises from fourth to third, carrying infrastructure and patch debt an incident count does not register, and SVC-05 falls because its incident volume comes with a clean infrastructure profile.

RQ2 asked to what extent operational service management metrics serve as valid proxies for technical debt severity. The answer is qualified: to a demonstrable but bounded extent, and the bound is worth stating exactly. The link between each indicator and a debt dimension is inherited from the literature rather than established here (Table 2.3). What this study adds is evidence that the composite built on those links behaves as an instrument should. The ranked profiles are differentiated rather than expressions of one underlying deficiency: SVC-06 carries severe debt across dimensions, SVC-01 quieter structural exposure behind a moderate incident record, SVC-04 the reverse, and SVC-03 neither. Indicators that moved together would have made the multi-criteria design redundant, and the discriminant test confirms that no single indicator reproduces the ordering. This is a necessary condition for the proxy claim, not a sufficient one. Criterion validity cannot be established, because no independent measure of non-code debt exists to validate against. The proxy relationship therefore keeps the status given in Chapter 1, a hypothesis that survived every test available within this design, not a proven law.

RQ3 asked how stable the rankings are under weight perturbation. They are stable at both extremes and concentrated at the top. The two discrete configurations agree closely in 2025 ($\tau = 0.867$). The Monte Carlo extension across 20,000 sampled weightings places SVC-03 last in 96.4% of them, while rank 1 falls to SVC-06 in 65.1% of draws and otherwise to SVC-04 or SVC-01. The single top service is therefore dominated by SVC-06 but weight-dependent within a stable three-service cluster while the lowest priority is weight-invariant. In 2024 the methods and weightings divide more evenly across the high-priority services, which, as Section 5.6 showed, reflects the more compressed 2024 profiles and marks the boundary between what the framework settles and what remains a managerial judgement.

6.3 Contribution to Knowledge

The contribution is threefold. First, to the best of current knowledge, this is the first application of TOPSIS to live ITSM and CMDB records for technical debt prioritisation in financial trading IT. It addresses the absence of validated quantitative approaches that Lenarduzzi *et al.* (2021) identified across the field. Second, the study contributes an operationally defined indicator set with an explicit literature mapping to debt dimensions, with evidence on how the set behaves under evaluation. Third, and less anticipated, the evaluation produced a methodological finding of independent interest. VIKOR's regret and the absolute-mode variant both refuse to reward a merely sample-extreme service the way vector-normalised TOPSIS does, and the 2024 data show this difference reordering the high-priority services. The resulting division of labour is a transferable design lesson beyond this case.

6.4 Practical Contribution

For the case organisation and similar institutions, the framework converts data already held in routine ITSM and CMDB records into an auditable priority ordering. Its primary use is portfolio oversight. Services are currently assessed one at a time, by different teams and on different evidence, so no consistent view of debt-related status exists across the portfolio. The ranking supplies one, and the profiles behind it show why each service sits where it does, letting portfolio managers and senior management argue remediation budgets and vendor conversations from a common picture rather than competing anecdotes. It also surfaces the debt current governance is most likely to miss. SVC-06 combines a heavy incident load with the portfolio's worst lifecycle and patch position, while SVC-01 looks comparatively quiet on incidents yet carries the highest MTTR and substantial infrastructure-lifecycle exposure once that exposure is measured per component rather than in aggregate.

The regulatory relevance is narrower than compliance. FINMA Circular 2023/1 places ICT risk management under board-level oversight (*FINMA Circular 2023/1 - Operational risks and resilience – banks*, 2024), and unsupported components and patch delay are ICT risks in that sense. The framework was not designed against

the circular. However, it provides a holistic overview of some of the exposures the institution must oversee in any case.

The framework is also transparent and reproducible by construction. Indicator definitions, normalisation, weights and computation are stated in full, with the implementation in Appendix A and the input data in Appendix B. Any reported ranking can be recomputed and independently verified. Proprietary static analysis tools, by contrast, rest on rulesets that users cannot inspect and that disagree on the same codebase (Amanatidis *et al.*, 2020). In a regulated setting, a verifiable ranking is easier to defend before auditors than an opaque score.

Two conditions govern its use. The scores are ordinal decision-support signals, not measurements of debt. They also describe services, not teams, since high-debt profiles typically reflect historical design decisions rather than current custodians (Ahmad *et al.*, 2026).

6.5 Limitations and Future Work

The limitations in Section 4.1.2 and Section 5.9 frame what remains to be done. The six-service demonstration set, selected for data completeness, limits the power of the rank-correlation tests and shares its dataset with the framework's construction, so the evaluation establishes robustness and method-independence rather than criterion validity. The design is cross-sectional. The CFR indicator shares its incident numerator with incident frequency, and recording-practice variation across teams can compress scores where informal incident handling is common. The method disagreement over the single top rank, documented in Section 5.3 and Section 5.6, adds a further boundary. The single top rank is dominated by SVC-06 but weight-dependent within the SVC-01, SVC-04 and SVC-06 cluster across the weight space, a compensability judgement between concentrated structural exposure and diffuse operational debt. The framework informs that judgement rather than settles it.

Four directions follow. The first is external validation in other regulated organisations, testing the proxy hypothesis beyond this case. The second is longitudinal

evaluation, tracking whether high-priority services go on to generate the consequences the debt literature predicts (Banker, Liang and Ramasubbu, 2021), which would move the proxy claim from plausible to evidenced. The third is portfolio-wide deployment once indicator coverage is complete, which would also strengthen the stability tests. The fourth is deriving the absolute-mode bounds from regulatory or cross-industry reference points rather than organisational thresholds, which would make its scores comparable between organisations as well as between years.

6.6 Reflective Note

Two aspects of this study turned out differently from what I expected. I anticipated that the main difficulty would be computational. It was instead definitional. Deciding what an incident count or a derived change failure rate can legitimately mean consumed more effort than implementing the model. I also expected the alternative methods to be a simple confirmation exercise. Instead, their disagreement over SVC-06 became one of the more instructive results, forcing an explicit position on whether debt dimensions are compensatory. The discipline of Design Science helped in both cases, by keeping the study honest about the difference between what the artefact demonstrates and what it merely suggests. The framework does not measure technical debt. It orders services by the operational shadow debt casts, and within the limits stated above, it does so reproducibly, defensibly and in a form a regulated institution can act on.

References

Ahmad, M.O., Mandić, V., Taušan, N., Katin, A. and Herath, P. (2026) 'Technical debt is not just technical: An industrial case study in large agile software development', *Journal of Systems and Software*, 234, p. 112719. Available at: https://doi.org/10.1016/j.jss.2025.112719.

Albarak, M., Bahsoon, R., Ozkaya, I. and Nord, R.L. (2022) 'Managing Technical Debt in Database Normalization', *IEEE Transactions on Software Engineering*, 48(3), pp. 755–772. Available at: https://doi.org/10.1109/TSE.2020.3001339.

Albuquerque, D., Guimarães, E., Tonin, G., Rodríguezs, P., Perkusich, M., Almeida, H., Perkusich, A. and Chagas, F. (2023) 'Managing Technical Debt Using Intelligent Techniques - A Systematic Mapping Study', *IEEE Transactions on Software Engineering*, 49(4), pp. 2202–2220. Available at: https://doi.org/10.1109/TSE.2022.3214764.

Alfayez, R., Winn, R., Alwehaibi, W., Venson, E. and Boehm, B. (2023) 'How SonarQube-identified technical debt is prioritized: An exploratory case study', *Information and Software Technology*, 156, p. 107147. Available at: https://doi.org/10.1016/j.infsof.2023.107147.

Amanatidis, T., Mittas, N., Moschou, A., Chatzigeorgiou, A., Ampatzoglou, A. and Angelis, L. (2020) 'Evaluating the agreement among technical debt measurement tools: Building an empirical benchmark of technical debt liabilities', *Empirical Software Engineering*, 25(5), pp. 4161–4204. Available at: https://doi.org/10.1007/s10664-020-09869-w.

Ampatzoglou, A., Bibi, S., Chatzigeorgiou, A., Avgeriou, P. and Stamelos, I.

(2018) 'Reusability Index: A Measure for Assessing Software Assets Reusability', in R. Capilla, B. Gallina, and C. Cetina (eds.) *New Opportunities for Software Reuse*. Cham: Springer International Publishing, pp. 43–58. Available at: https://doi.org/10.1007/978-3-319-90421-4_3.

Ampatzoglou, A., Chatzigeorgiou, A., Arvanitou, E.M. and Bibi, S. (2022) 'SDK4ED: A platform for technical debt management', *Software: Practice and Experience*, 52(8), pp. 1879–1902. Available at: https://doi.org/10.1002/spe.3093.

Avgeriou, P., Ozkaya, I., Chatzigeorgiou, A., Ciolkowski, M., Ernst, N.A., Koontz, R.J., Poort, E. and Shull, F. (2023) 'Technical Debt Management: The Road Ahead for Successful Software Delivery', *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, Melbourne, Australia: IEEE, pp. 15–30. Available at: https://doi.org/10.1109/ICSE-FoSE59343.2023.00007.

Banker, R., Liang, Y. and Ramasubbu, N. (2021) 'Technical Debt and Firm Performance', *Management Science*, 67(5), pp. 3174–3194. Available at: https://doi.org/10.1287/mnsc.2019.3542.

Bedi, J. and Kaur, K. (2022) 'Understanding factors affecting technical debt', *International Journal of Information Technology*, 14(2), pp. 1051–1060. Available at: https://doi.org/10.1007/s41870-020-00487-9.

Besker, T. (2020) *Technical Debt: An empirical investigation of its harmfulness and on management strategies in industry*. Chalmers University of Technology. Available at: https://research.chalmers.se/publication/518318/file/518318_Fulltext.pdf.

Biazotto, J.P., Feitosa, D., Avgeriou, P. and Nakagawa, E.Y. (2024) 'Technical debt management automation: State of the art and future perspectives', *Information and Software Technology*, 167, p. 107375. Available at: https://doi.org/10.1016/j.infsof.2023.107375.

Chén, O.Y., Bodelet, J.S., Saraiva, R.G., Phan, H., Di, J., Nagels, G., Schwantje, T., Cao, H., Gou, J., Reinen, J.M., Xiong, B., Zhi, B., Wang, X. and Vos, M. de (2023) 'The roles, challenges, and merits of the p value', *Patterns*, 4(12), p. 100878. Available at: https://doi.org/10.1016/j.patter.2023.100878.

Ciardiello, F. and Genovese, A. (2023) 'A comparison between TOPSIS and SAW methods', *Annals of Operations Research*, 325(2), pp. 967–994. Available at: https://doi.org/10.1007/s10479-023-05339-w.

Code of Conduct for BCS Members (2022). Swindon, UK: BCS, The Chartered Institute for IT, pp. 1–5. Available at: https://www.bcs.org/media/2211/bcs-code-of-conduct.pdf (Accessed: April 15, 2026).

Cunningham, W. (1992) 'The WyCash portfolio management system', *ACM SIGPLAN OOPS Messenger*, 4(2), pp. 29–30. Available at: https://doi.org/10.1145/157710.157715.

Federal Act on Data Protection (2023). Available at: https://www.fedlex.admin.ch/eli/cc/2022/491/en (Accessed: April 15, 2026).

FINMA Circular 2023/1 - Operational risks and resilience – banks (2024). Available at: https://www.finma.ch/en/~/media/finma/dokumente/dokumentencenter/myfinma/rundschreiben/finma-rs-2023-01-20221207.pdf (Accessed: March 29, 2026).

Forsgren, N., Humble, J. and Kim, G. (2018) *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland, OR: IT Revolution Press.

Fowler, M. (2009) *Technical Debt Quadrant*. Technical Debt Quadrant. Available at: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html (Accessed: March 29, 2026).

García-Cascales, M.S. and Lamata, M.T. (2012) 'On rank reversal and TOPSIS

method', *Mathematical and Computer Modelling*, 56(5), pp. 123–132. Available at: https://doi.org/10.1016/j.mcm.2011.12.022.

Greco, S., Ishizaka, A., Tasiou, M. and Torrisi, G. (2019) 'On the Methodological Framework of Composite Indices: A Review of the Issues of Weighting, Aggregation, and Robustness', *Social Indicators Research*, 141(1), pp. 61–94. Available at: https://doi.org/10.1007/s11205-017-1832-9.

Haki, K., Rieder, A., Buchmann, L. and Schneider, A.W. (2023) 'Digital nudging for technical debt management at Credit Suisse', *European Journal of Information Systems*, 32(1), pp. 64–80. Available at: https://doi.org/10.1080/0960085X.2022.2088413.

Hanssen, G.K., Brataas, G. and Martini, A. (2019) 'Identifying Scalability Debt in Open Systems', *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*. *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 48–52. Available at: https://doi.org/10.1109/TechDebt.2019.00014.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design Science in Information Systems Research', *MIS Quarterly*, 28(1), pp. 75–105. Available at: https://doi.org/10.2307/25148625.

Hwang, C.-L. and Yoon, K. (1981) *Multiple Attribute Decision Making*. Edited by M. Beckmann and H.P. Künzi. Berlin, Heidelberg: Springer (Lecture Notes in Economics and Mathematical Systems). Available at: https://doi.org/10.1007/978-3-642-48318-9.

Ishak, A. and Wanli (2020) 'Analysis of Fuzzy AHP-TOPSIS Methods in Multi Criteria Decision Making: Literature Review', *IOP Conference Series: Materials Science and Engineering*, 1003(1), p. 012147. Available at: https://doi.org/10.1088/1757-899X/1003/1/012147.

Kelly, L.M. and Cordeiro, M. (2020) 'Three principles of pragmatism for research on organizational processes', *Methodological Innovations*, 13(2), p.

2059799120937242. Available at: https://doi.org/10.1177/2059799120937242.

Khan, S. and Purohit, L. (2022) 'An Integrated Methodology of Ranking Based on PROMETHEE-CRITIC and TOPSIS-CRITIC In Web Service Domain', *2022 IEEE 11th International Conference on Communication Systems and Network Technologies (CSNT)*. *2022 IEEE 11th International Conference on Communication Systems and Network Technologies (CSNT)*, pp. 335–340. Available at: https://doi.org/10.1109/CSNT54456.2022.9787620.

Kleinwaks, H., Batchelor, A. and Bradley, T.H. (2023) 'Technical debt in systems engineering—A systematic literature review', *Systems Engineering*, 26(5), pp. 675–687. Available at: https://doi.org/10.1002/sys.21681.

Kruchten, P., Nord, R.L. and Ozkaya, I. (2012) 'Technical Debt: From Metaphor to Theory and Practice', *IEEE Software*, 29(6), pp. 18–21. Available at: https://doi.org/10.1109/MS.2012.167.

Kula, E., Rastogi, A., Huijgens, H., Deursen, A. van and Gousios, G. (2019) 'Releasing fast and slow: An exploratory case study at ING', *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. New York, NY, USA: Association for Computing Machinery (ESEC/FSE 2019), pp. 785–795. Available at: https://doi.org/10.1145/3338906.3338978.

Lenarduzzi, V., Besker, T., Taibi, D., Martini, A. and Arcelli Fontana, F. (2021) 'A systematic literature review on Technical Debt prioritization: Strategies, processes, factors, and tools', *Journal of Systems and Software*, 171, p. 110827. Available at: https://doi.org/10.1016/j.jss.2020.110827.

Markkanen, V. and Frantti, T. (2023) 'Patch management planning - towards one-to-one policy', *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*. *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*, pp. 60–69. Available at: https://doi.org/10.1109/DSA59317.2023.00018.

McConnell, S. (2008) 'Managing Technical Debt'. Construx Software. Available at: https://www.construx.com/wp-content/uploads/2019/02/CxWhitePaper_TechnicalDebt.pdf (Accessed: March 21, 2026).

Nardo, M., Saisana, M., Saltelli, A. and Tarantola, S. (2005) *Tools for Composite Indicators Building*. JRC Publications Repository. Available at: https://publications.jrc.ec.europa.eu/repository/handle/JRC31473 (Accessed: July 31, 2026).

Nielsen, M.E. and Skaarup, S. (2022) 'IT Portfolio management as a framework for managing Technical Debt: Theoretical framework applied on a case study', *14th International Conference on Theory and Practice of Electronic Governance*. *ICEGOV 2021: 14th International Conference on Theory and Practice of Electronic Governance*, New York, NY, USA: Association for Computing Machinery, pp. 89–96. Available at: https://doi.org/10.1145/3494193.3494296.

Nord, R.L., Ozkaya, I. and Woody, C. (2021) *Examples of Technical Debt's Cybersecurity Impact*. Pittsburgh, PA, USA: Carnegie Mellon University, pp. 1–21. Available at: https://apps.dtic.mil/sti/trecms/pdf/AD1144728.pdf (Accessed: April 27, 2026).

Nurse, J.R.C. (2025) 'To Patch or Not to Patch: Motivations, Challenges, and Implications for Cybersecurity', in A. Moallem (ed.) *HCI for Cybersecurity, Privacy and Trust*. Cham: Springer Nature Switzerland, pp. 265–281. Available at: https://doi.org/10.1007/978-3-031-92833-8_16.

Pandey, V., Komal and Dincer, H. (2023) 'A review on TOPSIS method and its extensions for different applications with recent development', *Soft Computing*, 27(23), pp. 18011–18039. Available at: https://doi.org/10.1007/s00500-023-09011-0.

Paudel, B., Gonzalez-Huerta, J., Zabardast, E. and Klotins, E. (2025) 'Exploring the Relationship Between Technical Debt and Lead Time: An Industrial Case Study', *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. *2025 IEEE International Conference on Software Analysis, Evolution and*

Reengineering (*SANER*), pp. 693–703. Available at: https://doi.org/10.1109/SANER64311.2025.00071.

Peffers, K., Tuunanen, T., Rothenberger, M.A. and Chatterjee, S. (2007) 'A Design Science Research Methodology for Information Systems Research', *Journal of Management Information Systems*, 24(3), pp. 45–77. Available at: https://doi.org/10.2753/MIS0742-1222240302.

Pina, D., Seaman, C. and Goldman, A. (2022) 'Technical Debt Prioritization: A Developer's Perspective', *2022 IEEE/ACM International Conference on Technical Debt (TechDebt)*. *2022 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 46–55. Available at: https://doi.org/10.1145/3524843.3528096.

Ramasubbu, N. and Kemerer, C.F. (2021) 'Controlling Technical Debt Remediation in Outsourced Enterprise Systems Maintenance: An Empirical Analysis', *Journal of Management Information Systems*, 38(1), pp. 4–28. Available at: https://doi.org/10.1080/07421222.2021.1870377.

Ramasubbu, N., Kemerer, C.F. and Woodard, C.J. (2015) 'Managing Technical Debt: Insights from Recent Empirical Evidence', *IEEE Software*, 32(2), pp. 22–25. Available at: https://doi.org/10.1109/MS.2015.45.

Rinta-Kahila, T., Penttinen, E., Aalto University, Lyytinen, K. and Case Western Reserve University (2023) 'Getting Trapped in Technical Debt: Sociotechnical Analysis of a Legacy System's Replacement', *MIS Quarterly*, 47(1), pp. 1–32. Available at: https://doi.org/10.25300/MISQ/2022/16711.

Rolland, K.-H. and Lyytinen, K. (2021) 'Exploring the Tensions Between Management of Architectural Debt and Digital Innovation: The Case of a Financial Organization', *Proceedings of the 54th Hawaii International Conference on System Sciences*. *54th Hawaii International Conference on System Sciences*, Hawaii International Conference on System Sciences, pp. 6722–6732. Available at: https://doi.org/10.24251/HICSS.2021.807.

Shekhovtsov, A. (2021) 'How strongly do rank similarity coefficients differ used in decision making problems?', *Procedia Computer Science*, 192, pp. 4570–4577. Available at: https://doi.org/10.1016/j.procs.2021.09.235.

Shekhovtsov, A. and Sałabun, W. (2020) 'A comparative case study of the VIKOR and TOPSIS rankings similarity', *Procedia Computer Science*, 176, pp. 3730–3740. Available at: https://doi.org/10.1016/j.procs.2020.09.014.

Stochel, M.G., Borek, T., Wawrowski, M.R. and Chołda, P. (2023) 'Business-driven technical debt management using Continuous Debt Valuation Approach (CoDVA)', *Information and Software Technology*, 164, p. 107333. Available at: https://doi.org/10.1016/j.infsof.2023.107333.

Thien, N.V., Dung, H.T. and Trung, D.D. (2024) 'Overcoming the Limitations of the RAPS Method by identifying Alternative Data Normalization Methods', *Engineering, Technology & Applied Science Research*, 14(4), pp. 15745–15750. Available at: https://doi.org/10.48084/etasr.7909.

Tizio, G.D., Armellini, M. and Massacci, F. (2023) 'Software Updates Strategies: A Quantitative Evaluation Against Advanced Persistent Threats', *IEEE Transactions on Software Engineering*, 49(3), pp. 1359–1373. Available at: https://doi.org/10.1109/TSE.2022.3176674.

de Toledo, S.S., Martini, A., Sjøberg, D.I.K., Przybyszewska, A. and Frandsen, J.S. (2021) 'Reducing Incidents in Microservices by Repaying Architectural Technical Debt'. *2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 196–205. Available at: https://doi.org/10.1109/SEAA53835.2021.00033.

Tornhill, A. and Borg, M. (2022) 'Code Red: The Business Impact of Code Quality - A Quantitative Study of 39 Proprietary Production Codebases', *2022 IEEE/ACM International Conference on Technical Debt (TechDebt). 2022 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 11–20. Available at: https://doi.org/10.1145/3524843.3528091.

Turnbull, D., Chugh, R. and Luck, J. (2021) 'The Use of Case Study Design in Learning Management System Research: A Label of Convenience?', *International Journal of Qualitative Methods*, 20, pp. 1–11. Available at: https://doi.org/10.1177/16094069211004148.

Vidoni, M., Codabux, Z. and Fard, F.H. (2022) 'Infinite technical debt', *Journal of Systems and Software*, 190, p. 111336. Available at: https://doi.org/10.1016/j.jss.2022.111336.

Wiese, M. and Borowa, K. (2023) 'IT managers' perspective on Technical Debt Management', *Journal of Systems and Software*, 202, p. 111700. Available at: https://doi.org/10.1016/j.jss.2023.111700.

Yang, W. (2020) 'Ingenious Solution for the Rank Reversal Problem of TOPSIS Method', *Mathematical Problems in Engineering*, 2020(1), p. 9676518. Available at: https://doi.org/10.1155/2020/9676518.

Yang, Y., Michel, R., Wade, J., Verma, D., Törngren, M. and Alelyani, T. (2019) 'Towards a taxonomy of technical debt for COTS-intensive cyber physical systems', *Procedia Computer Science*, 153, pp. 108–117. Available at: https://doi.org/10.1016/j.procs.2019.05.061.

Zanakis, S.H., Solomon, A., Wishart, N. and Dublish, S. (1998) 'Multi-attribute decision making: A simulation comparison of select methods', *European Journal of Operational Research*, 107(3), pp. 507–529. Available at: https://doi.org/10.1016/S0377-2217(97)00147-1.

Zytoon, M.A. (2020) 'A Decision Support Model for Prioritization of Regulated Safety Inspections Using Integrated Delphi, AHP and Double-Hierarchical TOPSIS Approach', *IEEE Access*, 8, pp. 83444–83464. Available at: https://doi.org/10.1109/ACCESS.2020.2991179.

A Appendix A: Python Code

The content in this appendix is the exact artefact which has been used to calculate the TOPSIS rankings. Further artefacts which have been used to do other calculations can be found on the author's GitHub repository: https://github.com/TobiZeier/MScProject.

```
#!/usr/bin/env python
"""
This file is part of the master's thesis with the title Measuring
 ↪  Technical Debt in Mission Critical Trading Systems.

It carries several functionalities such as:
- TOPSIS ranking in classic mode with two different weight settings
- TOPSIS ranking in absolute mode with fixed bounds and a pre
 ↪  defined best/worst alternative
- Graphical export of results

running command: python topsis_ranking.py services2025.csv --mode
 ↪  classic --weights equal
running command: python topsis_ranking.py services2025.csv --mode
 ↪  absolute --weights equal
"""

import argparse
import sys
import re

import numpy as np
```

```
import pandas as pd
from pathlib import Path
import plotly.graph_objects as go

CRITERIA = [
    "incident_freq",
    "mttr_hrs",
    "cfr",
    "patch_recency",
    "unsupported_months"
]

COST_CRITERIA = [
    "incident_freq",
    "mttr_hrs",
    "cfr",
    "unsupported_months"
]

#--- two different weight settings
 ↪  ------------------------------------------------------------------
EQUAL_WEIGHTS = np.array([0.2, 0.2, 0.2, 0.2, 0.2], dtype=float)
ADJUSTED_WEIGHTS = np.array([0.4/3, 0.3, 0.4/3, 0.3, 0.4/3],
 ↪  dtype=float)

#--- fixed bounds to prevent rank reversal
 ↪  ------------------------------------------------------------
ABSOLUTE_BOUNDS = {
    "incident_freq": (0.0, 1000.0),
    "mttr_hrs": (0.0, 120.0),
    "cfr": (0.0, 5.0),
    "patch_recency": (0.0 , 100.0),
    "unsupported_months": (0.0, 30.0)
```

```
}


#--- classic TOPSIS ranking
 ↪  ------------------------------------------------------------------------
def topsis_classic(df, weights, criteria = CRITERIA, cost_criteria
 ↪  = COST_CRITERIA):
    _check_weights(weights)

    X = df[criteria].to_numpy(dtype=float)

    norms = np.sqrt((X ** 2).sum(axis=0))
    if np.any(norms == 0):
        raise ValueError("One or more criterion columns are all
         ↪  zeros - normalisation not possible.")
    R = X / norms
    V = R * weights

    benefit = np.array([c not in cost_criteria for c in criteria])
    ideal_best = np.where(benefit, V.max(axis=0), V.min(axis=0))
    ideal_worst = np.where(benefit, V.min(axis=0), V.max(axis=0))

    d_best = np.sqrt(((V - ideal_best) ** 2).sum(axis=1))
    d_worst = np.sqrt(((V - ideal_worst) ** 2).sum(axis=1))

    total = d_best + d_worst
    if np.any(total == 0):
        raise ValueError("A row is equidistant from both ideals -
         ↪  closeness coefficient is undefined.")
    return d_worst / total


#--- absolute TOPSIS ranking
 ↪  ------------------------------------------------------------------
```

```
def topsis_absolute(df, weights, criteria = CRITERIA, cost_criteria
 ↪  = COST_CRITERIA, bounds=ABSOLUTE_BOUNDS):
    _check_weights(weights)

    X = df[criteria].to_numpy(dtype=float)

    lo = np.array([bounds[c][0] for c in criteria], dtype=float)
    hi = np.array([bounds[c][1] for c in criteria], dtype=float)
    if np.any(hi <= lo):
        raise ValueError("Each criterion bound must satisfy upper >
         ↪  lower.")
    if np.any(X > hi) or np.any(X < lo):
        raise ValueError("An observed value lies outside its fixed
         ↪  criterion bounds; widen the bounds.")

    benefit = np.array([c not in cost_criteria for c in criteria])

    # Max normalisation against the fixed ceiling (Garcia-Cascales:
     ↪  n = z / max).
    R = X / hi

    V = R * weights

    best_raw = np.where(benefit, hi, lo)
    worst_raw = np.where(benefit, lo, hi)
    ideal_best = (best_raw / hi) * weights
    ideal_worst = (worst_raw / hi) * weights

    d_best = np.sqrt(((V - ideal_best) ** 2).sum(axis=1))
    d_worst = np.sqrt(((V - ideal_worst) ** 2).sum(axis=1))

    total = d_best + d_worst
    if np.any(total == 0):
```

```
        raise ValueError("A row is equidistant from both ideals -
         ↪ closeness coefficient is undefined.")
    return d_worst / total



SCORERS = {"classic": topsis_classic, "absolute": topsis_absolute}

#--- printing results and charts
 ↪ ------------------------------------------------------------

def print_results(df, weights, scorer, mode, year, weight_label):
    result = rank_table(df, scorer(df, weights))
    title = f"TOPSIS ranking ({year}, {weight_label} weights,
 ↪ {mode} mode)"
    print("\n" + "=" * 70)
    print(f"  {title}")
    print("=" * 70)
    print(f"  {'Service':<12} {'Closeness Coeff (Ci)':>22}
     ↪ {'Rank':>6}")
    print("-" * 70)
    for _, row in result.iterrows():
        print(f"  {row['service']:<12}
         ↪ {row['closeness_coeff']:>22.4f} {row['rank']:>6}")
    print("=" * 70)
    print("  Rank 1 = highest TD priority  |  Ci near 0 = worst
     ↪ profile\n")



def save_chart(result, output_path, subtitle):
    df_plot = result.sort_values(["rank", "closeness_coeff"],
 ↪ ascending=[True, False]).reset_index(drop=True)
    df_plot["label"] = df_plot.apply(lambda r: f"Rank
 ↪ {int(r['rank'])} · {r['closeness_coeff']:.4f}", axis=1)
```

```
    colors = ["#c0392b" if r == 1 else "#2471a3" for r in
↪   df_plot["rank"]]

    fig = go.Figure(go.Bar(
        y=df_plot["service"],
        x=df_plot["closeness_coeff"],
        orientation="h",
        marker_color=colors,
        text=df_plot["label"],
        textposition="outside",
        cliponaxis=False,
        insidetextanchor="middle",
    ))

    fig.update_layout(
        width=1100,
        height=max(420, 95 * len(df_plot) + 170),
        title=dict(
            text=(
                "Technical Debt Priority<br>"
                f"<span
                 ↪  style='font-size:15px;font-weight:normal;'>{subtitle}</span>"
            ),
            font=dict(size=19),
            y=0.93,
        ),
        plot_bgcolor="white",
        paper_bgcolor="white",
        margin=dict(t=140, b=60, l=130, r=90),
        xaxis=dict(
            title="Closeness coefficient Ci",
            range=[0, 1.05],
```

```
            tickformat=".2f",
            gridcolor="#dddddd",
            zeroline=False,
            tickfont=dict(size=13),
        ),
        yaxis=dict(
            title="Service",
            autorange="reversed",
            tickfont=dict(size=13),
        ),
        showlegend=False,
        uniformtext_minsize=11,
        uniformtext_mode="hide",
    )

    try:
        fig.write_image(output_path)
    except Exception as exc:
        raise RuntimeError(
            "Chart export failed - is kaleido installed? Try: pip
             ↪  install kaleido"
        ) from exc

    print(f"Chart saved: {output_path}")


#--- input handling
 ↪  ------------------------------------------------------------------------
def _check_weights(weights):
    if not np.isclose(weights.sum(), 1.0):
        raise ValueError(f"Weights should sum to 1.0, got
         ↪  {weights.sum():.4f}")
```

```
def rank_table(df, closeness):
    out = df[["service"]].copy()
    out["closeness_coeff"] = closeness
    out["rank"] = out["closeness_coeff"].rank(ascending=True,
 ↪  method="min").astype(int)
    return out.sort_values(["rank", "closeness_coeff",
     ↪  "service"]).reset_index(drop=True)

def load_csv(path):
    try:
        df = pd.read_csv(path)
    except FileNotFoundError:
        sys.exit(f"File not found: {path}")

    needed = {"service"} | set(CRITERIA)
    missing = needed - set(df.columns)
    if missing:
        sys.exit(f"CSV is missing these columns:
 ↪  {sorted(missing)}")

    for col in CRITERIA:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    bad = [c for c in CRITERIA if df[c].isnull().any()]
    if bad:
        sys.exit(f"Non-numeric or missing values found in: {bad}")

    if df["service"].isnull().any():
        sys.exit("Some rows in the service column are empty.")
    return df


def main():
```

```
    parser = argparse.ArgumentParser(description=__doc__,
↪   formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("csv_file", nargs="?", help="Path to the
↪   input CSV file")
    parser.add_argument("--mode", choices=["classic", "absolute"],
↪   default="absolute",
                        help="Scoring variant (default: absolute,
                         ↪  the rank-reversal-safe method)")
    parser.add_argument("--weights", choices=["equal", "adjusted"],
↪   default="equal",
                        help="Weight configuration for the point
                         ↪  ranking and reversal test")
    args = parser.parse_args()

    df = load_csv(args.csv_file)

    match = re.search(r"\d{4}", Path(args.csv_file).stem)
    year = match.group() if match else "unknown"

    scorer = SCORERS[args.mode]
    weights = EQUAL_WEIGHTS if args.weights == "equal" else
↪   ADJUSTED_WEIGHTS

    result = rank_table(df, scorer(df, weights))
    output_path =
↪   f"topsis_ranking_{year}_{args.mode}_{args.weights}.png"

    print_results(df, weights, scorer, args.mode, year,
↪   args.weights)
    save_chart(result, output_path, f"{args.mode} mode,
↪   {args.weights} weights, {year}")
```

```
if __name__ == '__main__':
    main()
```

B Appendix B: Data

This appendix contains the contents of the CSV files that were used as input for the TOPSIS calculations presented in Chapter 4.

Filename: `services2025.csv`

Filesize: `262 bytes`

```
service,incident_freq,mttr_hrs,cfr,patch_recency,unsupported_months
SVC-01,64,45.35,0.83,53.10,5.6748
SVC-02,30,25.26,0.19,61.90,2.4218
SVC-03,21,13.31,0.38,100.00,0
SVC-04,574,41.91,3.38,100.00,0
SVC-05,171,23.72,1.52,100.00,0
SVC-06,302,13.11,3.96,52.94,18.8824
```

Filename: `services2024.csv`

Filesize: `262 bytes`

```
service,incident_freq,mttr_hrs,cfr,patch_recency,unsupported_months
SVC-01,65,55.37,0.29,98.23,0.2389
SVC-02,39,81.35,0.11,99.66,0.1156
SVC-03,63,9.77,0.86,100.00,0
SVC-04,410,41.53,2.19,100.00,0
SVC-05,140,30.30,1.41,100.00,0
SVC-06,43,15.87,0.86,82.35,16.4118
```

C Appendix C: Full Evaluation Results

This appendix records the complete output behind the summary tables in Chapter 5. It lists, for every weight configuration and both observation years, the per-service scores produced by each method, together with the full convergent, discriminant, rank-reversal, weight-space and temporal results. Absolute-mode results are reported alongside the standard formulation as the rank-reversal-safe variant discussed in Section 5.5. In the score tables, C_i is the TOPSIS closeness coefficient, SAW is the Simple Additive Weighting score, and S, R and Q are the VIKOR utility, regret and compromise measures. The final column gives each service's rank under TOPSIS, SAW and VIKOR in that order, where rank 1 is the highest debt priority. Rows are ordered by TOPSIS rank.

C.1 Per-Service Scores and Rankings (Classic Mode)

Table C.1: Full scores and rankings, 2025 equal weights.

Service	C_i	SAW	S	R	Q	Rank (T, S, V)
SVC-06	0.3204	0.2984	0.7016	0.2000	1.0000	1, 1, 1
SVC-04	0.4778	0.4521	0.5479	0.2000	0.8886	2, 2, 2
SVC-01	0.6619	0.4911	0.5089	0.2000	0.8604	3, 3, 3
SVC-05	0.7790	0.8094	0.1906	0.0706	0.2891	4, 5, 5
SVC-02	0.8297	0.7338	0.2662	0.1619	0.5844	5, 4, 4
SVC-03	0.9774	0.9887	0.0113	0.0101	0.0000	6, 6, 6

Table C.2: Full scores and rankings, 2025 adjusted weights.

Service	C_i	SAW	S	R	Q	Rank (T, S, V)
SVC-06	0.4415	0.3656	0.6344	0.3000	0.9716	1, 2, 2
SVC-04	0.4512	0.4859	0.5141	0.2680	0.8264	2, 3, 3
SVC-01	0.4976	0.3279	0.6721	0.3000	1.0000	3, 1, 1
SVC-02	0.7175	0.6248	0.3752	0.2429	0.6789	4, 4, 4
SVC-05	0.7615	0.8181	0.1819	0.0987	0.2875	5, 5, 5
SVC-03	0.9809	0.9914	0.0086	0.0067	0.0000	6, 6, 6

Table C.3: Full scores and rankings, 2024 equal weights.

Service	C_i	SAW	S	R	Q	Rank (T, S, V)
SVC-04	0.4837	0.5113	0.4887	0.2000	0.9968	1, 2, 2
SVC-06	0.5163	0.5087	0.4913	0.2000	1.0000	2, 1, 1
SVC-02	0.6999	0.7947	0.2053	0.2000	0.6479	3, 4, 3
SVC-05	0.7058	0.7632	0.2368	0.1250	0.3935	4, 3, 4
SVC-01	0.7759	0.8183	0.1817	0.1274	0.3351	5, 5, 5
SVC-03	0.8492	0.9149	0.0851	0.0721	0.0000	6, 6, 6

Table C.4: Full scores and rankings, 2024 adjusted weights.

Service	C_i	SAW	S	R	Q	Rank (T, S, V)
SVC-04	0.5038	0.6002	0.3998	0.1333	0.5490	1, 2, 3
SVC-02	0.5102	0.6933	0.3067	0.3000	0.7767	2, 3, 2
SVC-06	0.6048	0.4916	0.5084	0.3000	1.0000	3, 1, 1
SVC-01	0.6231	0.7560	0.2440	0.1911	0.4912	4, 4, 4
SVC-05	0.7092	0.7943	0.2057	0.0860	0.2403	5, 5, 5
SVC-03	0.8816	0.9433	0.0567	0.0481	0.0000	6, 6, 6

C.2 Per-Service Scores and Rankings (Absolute Mode)

Under absolute-mode TOPSIS the 2025 equal-weight agreement with SAW and VIKOR is $\tau = 1.000$: the variant orders SVC-02 above SVC-05 where the standard formulation reverses that adjacent pair, aligning it exactly with both compensatory methods. It accompanies the rank-reversal results in Table C.10.

Table C.5: Full scores and rankings under absolute-mode TOPSIS, 2025 equal weights.

Service	C_i	SAW	S	R	Q	Rank (T, S, V)
SVC-06	0.5320	0.2984	0.7016	0.2000	1.0000	1, 1, 1
SVC-04	0.6333	0.4521	0.5479	0.2000	0.8886	2, 2, 2
SVC-01	0.7219	0.4911	0.5089	0.2000	0.8604	3, 3, 3
SVC-02	0.8124	0.7338	0.2662	0.1619	0.5844	4, 4, 4
SVC-05	0.8297	0.8094	0.1906	0.0706	0.2891	5, 5, 5
SVC-03	0.9404	0.9887	0.0113	0.0101	0.0000	6, 6, 6

C.3 Convergent Agreement (Classic Mode)

Table C.6: Rank agreement (Kendall's τ) between classic TOPSIS and the alternative methods.

Configuration	TOPSIS vs SAW	TOPSIS vs VIKOR
2025, equal	0.867	0.867
2025, adjusted	0.733	0.733
2024, equal	0.733	0.867
2024, adjusted	0.733	0.600

C.4 Convergent Agreement (Absolute Mode)

Table C.7: Rank agreement (Kendall's τ) between absolute TOPSIS and the alternative methods.

Configuration	TOPSIS vs SAW	TOPSIS vs VIKOR
2025, equal	1.000	1.000
2025, adjusted	0.867	0.867
2024, equal	0.600	0.733
2024, adjusted	0.333	0.467

C.5 Discriminant Detail

Table C.8: TOPSIS ranking against a ranking by incident frequency alone, equal weights.

Year	TOPSIS order	Incident-only order	τ
2025	SVC-06 $\succ$ SVC-04 $\succ$ SVC-01 $\succ$ SVC-05 $\succ$ SVC-02 $\succ$ SVC-03	SVC-04 $\succ$ SVC-06 $\succ$ SVC-05 $\succ$ SVC-01 $\succ$ SVC-02 $\succ$ SVC-03	0.733
2024	SVC-04 $\succ$ SVC-06 $\succ$ SVC-02 $\succ$ SVC-05 $\succ$ SVC-01 $\succ$ SVC-03	SVC-04 $\succ$ SVC-05 $\succ$ SVC-01 $\succ$ SVC-03 $\succ$ SVC-06 $\succ$ SVC-02	0.200

C.6 Rank-Reversal Detail (Classic Mode)

Entries reading preserved leave the reference order unchanged. The 2025 reference order is SVC-06 $\succ$ SVC-04 $\succ$ SVC-01 $\succ$ SVC-05 $\succ$ SVC-02 $\succ$ SVC-03. The 2024 reference order is SVC-04 $\succ$ SVC-06 $\succ$ SVC-02 $\succ$ SVC-05 $\succ$ SVC-01 $\succ$ SVC-03.

Table C.9: Leave-one-out rank order, standard TOPSIS, equal weights.

Service removed	2025 outcome	2024 outcome
SVC-01	preserved	preserved
SVC-02	preserved	preserved

Service removed	2025 outcome	2024 outcome
SVC-03	preserved	SVC-04 ≻ SVC-06 ≻ SVC-05 ≻ SVC-02 ≻ SVC-01
SVC-04	preserved	SVC-06 ≻ SVC-05 ≻ SVC-02 ≻ SVC-01 ≻ SVC-03
SVC-05	preserved	preserved
SVC-06	preserved	SVC-04 ≻ SVC-01 ≻ SVC-02 ≻ SVC-05 ≻ SVC-03

C.7 Rank-Reversal Detail (Absolute Mode)

Reference orders: 2025 SVC-06 ≻ SVC-04 ≻ SVC-01 ≻ SVC-02 ≻ SVC-05 ≻ SVC-03; 2024 SVC-04 ≻ SVC-06 ≻ SVC-02 ≻ SVC-01 ≻ SVC-05 ≻ SVC-03.

TABLE C.10: Leave-one-out rank order, absolute-mode TOPSIS, equal weights

Service removed	2025 outcome	2024 outcome
SVC-01	preserved	preserved
SVC-02	preserved	preserved
SVC-03	preserved	preserved
SVC-04	preserved	preserved
SVC-05	preserved	preserved
SVC-06	preserved	preserved

C.8 Weight-Space Robustness Detail

The 2025 rank-acceptability distribution is reported in Table 5.3. The corresponding 2024 distribution, summarised in Section 5.7, is given below.

TABLE C.11: Rank-acceptability distribution, absolute-mode TOPSIS, 2024 (20,000 weight samples, seed 50); cells are rank probabilities

Service	R1	R2	R3	R4	R5	R6
SVC-01	0.0000	0.1334	0.2136	0.2308	0.3040	0.1183
SVC-02	0.2507	0.2310	0.1567	0.1165	0.0891	0.1560
SVC-03	0.0000	0.0000	0.0046	0.2072	0.0679	0.7202
SVC-04	0.4262	0.2971	0.1889	0.0878	0.0000	0.0000
SVC-05	0.0000	0.1646	0.2722	0.2596	0.3036	0.0000
SVC-06	0.3230	0.1740	0.1640	0.0981	0.2354	0.0055

C.9 Temporal Stability

TABLE C.12: TOPSIS rankings across the two observation years.

Configuration	2025 order	2024 order	τ
Equal	SVC-06 $\succ$ SVC-04 $\succ$ SVC-01 $\succ$ SVC-05 $\succ$ SVC-02 $\succ$ SVC-03	SVC-04 $\succ$ SVC-06 $\succ$ SVC-02 $\succ$ SVC-05 $\succ$ SVC-01 $\succ$ SVC-03	0.467
Adjusted	SVC-06 $\succ$ SVC-04 $\succ$ SVC-01 $\succ$ SVC-02 $\succ$ SVC-05 $\succ$ SVC-03	SVC-04 $\succ$ SVC-02 $\succ$ SVC-06 $\succ$ SVC-01 $\succ$ SVC-05 $\succ$ SVC-03	0.600